

UNIT 4

AIML

Logistic regression is a supervised machine learning algorithm that accomplishes binary classification tasks by predicting the probability of an outcome, event, or observation. The model delivers a binary or dichotomous outcome limited to two possible outcomes: yes/no, 0/1, or true/false.

Logistical regression analyzes the relationship between one or more independent variables and classifies data into discrete classes. It is extensively used in [predictive modeling](#), where the model estimates the mathematical probability of whether an instance belongs to a specific category or not.

For example, 0 – represents a negative class; 1 – represents a positive class. Logistic regression is commonly used in binary classification problems where the outcome variable reveals either of the two categories (0 and 1).

Some examples of such classifications and instances where the binary response is expected or implied are:

- 1. Determine the probability of heart attacks:** With the help of a logistic model, medical practitioners can determine the relationship between variables such as the weight, exercise, etc., of an individual and use it to predict whether the person will suffer from a heart attack or any other medical complication.
- 2. Possibility of enrolling into a university:** Application aggregators can determine the probability of a student getting accepted to a particular university or a degree course in a college by studying the relationship between the estimator variables, such as GRE, GMAT, or TOEFL scores.
- 3. Identifying spam emails:** Email inboxes are filtered to determine if the email communication is promotional/spam by understanding the predictor variables and applying a logistic regression algorithm to check its authenticity.

Key advantages of logistic regression

The logistic regression analysis has several advantages in the field of machine learning.

1. Easier to implement machine learning methods: A [machine learning model](#) can be effectively set up with the help of training and testing. The training identifies patterns in the input data (image) and associates them with some form of output (label). Training a logistic model with a regression algorithm does not demand higher computational power. As such, logistic regression is easier to implement, interpret, and train than other ML methods.

2. Suitable for linearly separable datasets: A linearly separable dataset refers to a graph where a straight line separates the two data classes. In logistic regression, the y variable takes only two values. Hence, one can effectively classify data into two separate classes if linearly separable data is used.

3. Provides valuable insights: Logistic regression measures how relevant or appropriate an independent/predictor variable is (coefficient size) and also reveals the direction of their relationship or association (positive or negative).

Logistic Regression Equation and Assumptions

Logistic regression uses a logistic function called a sigmoid function to map predictions and their probabilities. The sigmoid function refers to an S-shaped curve that converts any real value to a range between 0 and 1.

Moreover, if the output of the sigmoid function (estimated probability) is greater than a predefined threshold on the graph, the model predicts that the instance belongs to that class. If the estimated probability is less than the predefined threshold, the model predicts that the instance does not belong to the class.

For example, if the output of the sigmoid function is above 0.5, the output is considered as 1. On the other hand, if the output is less than 0.5, the output is classified as 0. Also, if the graph goes further to the negative end, the predicted value of y will be 0 and vice versa. In other words, if the output of

the sigmoid function is 0.65, it implies that there are 65% chances of the event occurring; a coin toss, for example.

The sigmoid function is referred to as an activation function for logistic regression and is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Equation of Logistic Regression

where,

- e = base of natural logarithms
- value = numerical value one wishes to transform

The following equation represents logistic regression:

$$y = \frac{e^{(b_0 + b_1x)}}{1 + e^{(b_0 + b_1x)}}$$

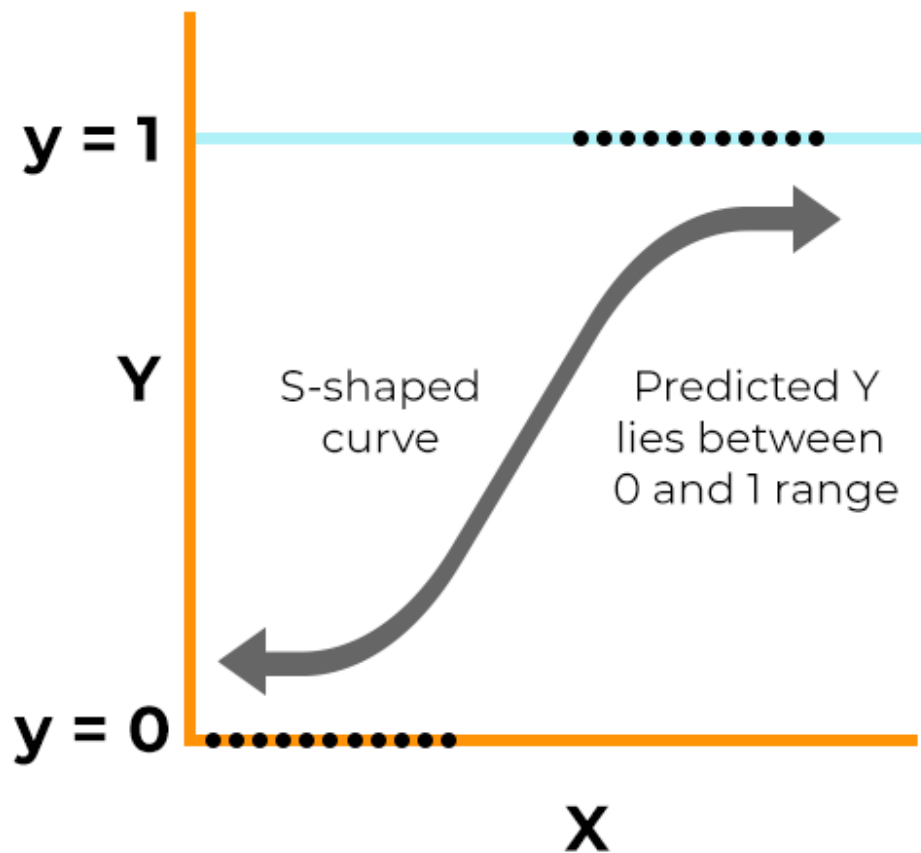
Logistic Regression – Sigmoid Function

Here,

- x = input value
- y = predicted output
- b0 = bias or intercept term
- b1 = coefficient for input (x)

This equation is similar to linear regression, where the input values are combined linearly to predict an output value using weights or coefficient values. However, unlike linear regression, the output value modeled here is a binary value (0 or 1) rather than a numeric value.

Logistic Regression



Key Assumptions for Implementing Logistic Regression

Key properties of the logistic regression equation

Typical properties of the logistic regression equation include:

- Logistic regression's dependent variable obeys 'Bernoulli distribution'
- Estimation/prediction is based on 'maximum likelihood.'
- Logistic regression does not evaluate the coefficient of determination (or R squared) as observed in linear regression'. Instead, the model's fitness is assessed through a concordance.

For example, KS or Kolmogorov-Smirnov statistics look at the difference between cumulative events and cumulative non-events to determine the efficacy of models through credit scoring.

Types of Logistic Regression with Examples

Logistic regression is classified into binary, multinomial, and ordinal. Each type differs from the other in execution and theory. Let's understand each type in detail.

1. Binary logistic regression

Binary logistic regression predicts the relationship between the independent and binary dependent variables. Some examples of the output of this regression type may be, success/failure, 0/1, or true/false.

Examples:

1. Deciding on whether or not to offer a loan to a bank customer:
Outcome = yes or no.
2. Evaluating the risk of cancer: Outcome = high or low.
3. Predicting a team's win in a football match: Outcome = yes or no.

2. Multinomial logistic regression

A categorical dependent variable has two or more discrete outcomes in a multinomial regression type. This implies that this regression type has more than two possible outcomes.

Examples:

1. Let's say you want to predict the most popular transportation type for 2040. Here, transport type equates to the dependent variable, and the possible outcomes can be electric cars, electric trains, electric buses, and electric bikes.
2. Predicting whether a student will join a college, vocational/trade school, or corporate industry.
3. Estimating the type of food consumed by pets, the outcome may be wet food, dry food, or junk food.

3. Ordinal logistic regression

Ordinal logistic regression applies when the dependent variable is in an ordered state (i.e., ordinal). The dependent variable (y) specifies an order with two or more categories or levels.

Examples: Dependent variables represent,

1. Formal shirt size: Outcomes = XS/S/M/L/XL
2. Survey answers: Outcomes = Agree/Disagree/Unsure
3. Scores on a math test: Outcomes = Poor/Average/Good

Perceptron in Machine Learning

In Machine Learning and Artificial Intelligence, Perceptron is the most commonly used term for all folks. It is the primary step to learn Machine Learning and Deep Learning technologies, which consists of a set of weights, input values or scores, and a threshold. ***Perceptron is a building block of an Artificial Neural Network.*** Initially, in the mid of 19th century, **Mr. Frank Rosenblatt** invented the Perceptron for performing certain calculations to detect input data capabilities or business intelligence. Perceptron is a linear Machine Learning algorithm used for supervised learning for various binary classifiers. This algorithm enables neurons to learn elements and processes them one by one during preparation.

What is the Perceptron model in Machine Learning?

Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, ***Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence.***

Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

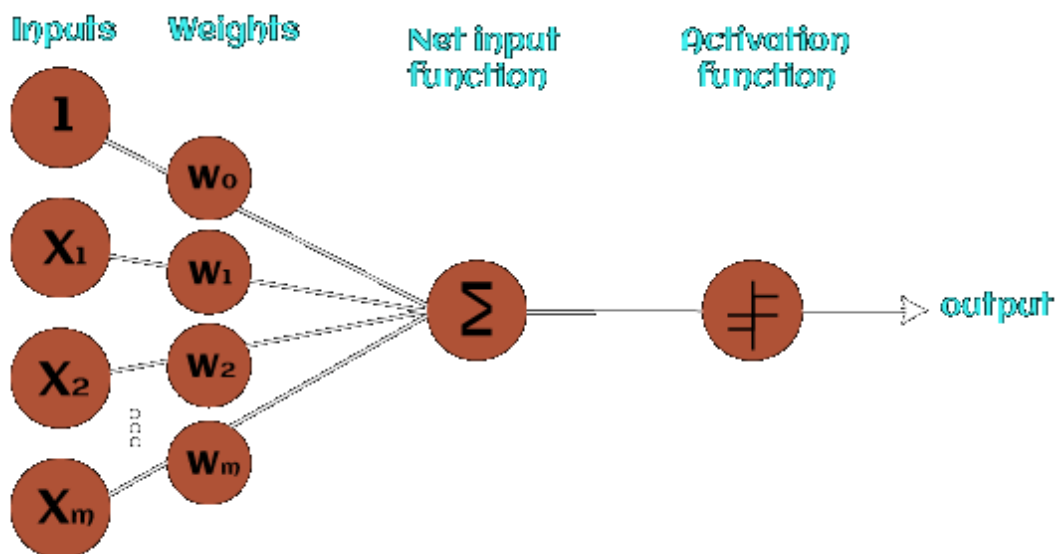
What is Binary classifier in Machine Learning?

In Machine Learning, binary classifiers are defined as the function that helps in deciding whether input data can be represented as vectors of numbers and belongs to some specific class.

Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a **classification algorithm that can predict linear predictor function in terms of weight and feature vectors.**

Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:



- **Input Nodes or Input Layer:**

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

- **Weight and Bias:**

Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly

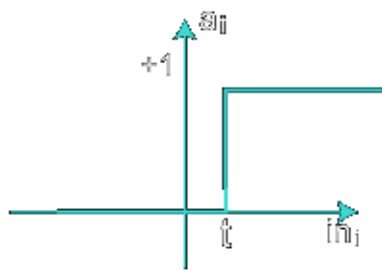
proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

- **Activation Function:**

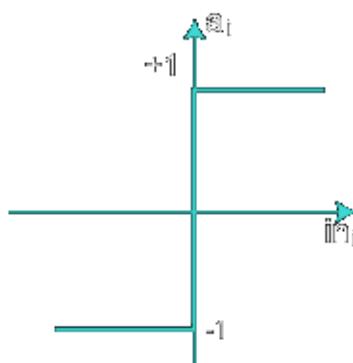
These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

Types of Activation functions:

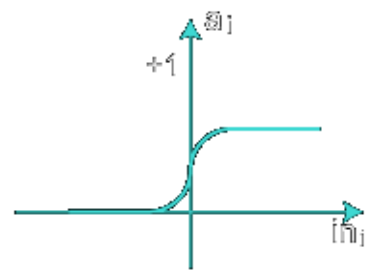
- Sign function
- Step function, and
- Sigmoid function



Step Function



Sign Function

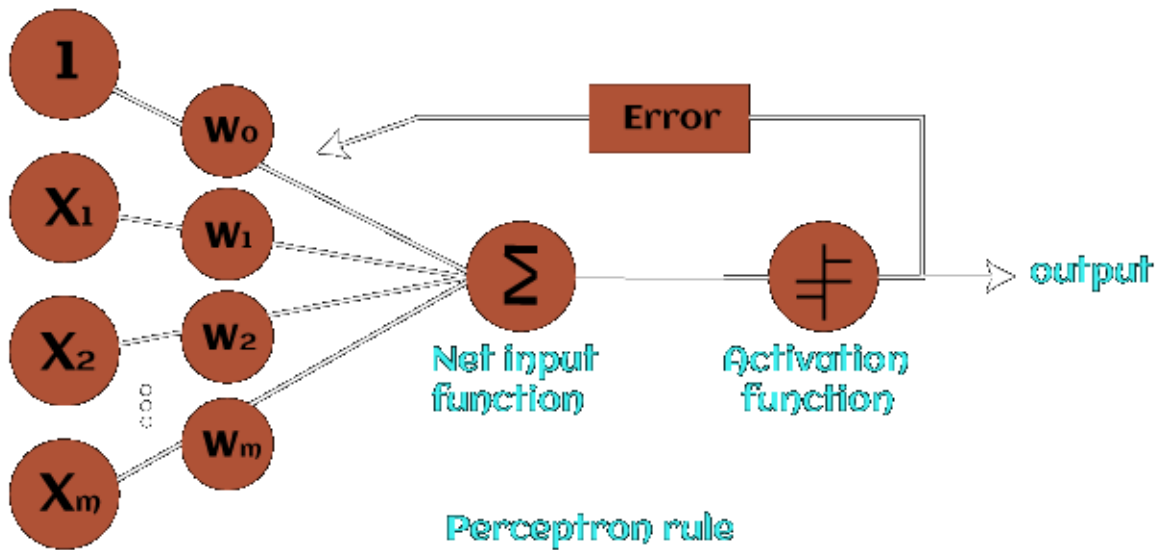


Sigmoid Function

The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.

How does Perceptron work?

In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the **step function** and is represented by 'f'.



This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

Perceptron model works in two important steps as follows:

Step-1

In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots w_n * x_n$$

Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.

$$\sum w_i * x_i + b$$

Step-2

In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

$$Y = f(\sum w_i * x_i + b)$$

Types of Perceptron Models

Based on the layers, Perceptron models are divided into two types. These are as follows:

1. Single-layer Perceptron Model
2. Multi-layer Perceptron model

Single Layer Perceptron Model:

This is one of the easiest artificial neural networks (ANN) types. A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.

In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters. Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

If the outcome is same as pre-determined or threshold value, then the performance of this model is stated as satisfied, and weight demand does not change. However, this model consists of a few discrepancies triggered when multiple weight inputs values are fed into the model. Hence, to find desired output and minimize errors, some changes should be necessary for the weights input.

"Single-layer perceptron can learn only linearly separable patterns."

Multi-Layered Perceptron Model:

Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a greater number of hidden layers.

The multi-layer perceptron model is also known as the Back propagation algorithm, which executes in two stages as follows:

- **Forward Stage:** Activation functions start from the input layer in the forward stage and terminate on the output layer.

- **Backward Stage:** In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

Hence, a multi-layered perceptron model has considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.

A multi-layer perceptron model has greater processing power and can process linear and non-linear patterns. Further, it can also implement logic gates such as AND, OR, XOR, NAND, NOT, XNOR, NOR.

Advantages of Multi-Layer Perceptron:

- A multi-layered perceptron model can be used to solve complex non-linear problems.
- It works well with both small and large input data.
- It helps us to obtain quick predictions after the training.
- It helps to obtain the same accuracy ratio with large as well as small data.

Disadvantages of Multi-Layer Perceptron:

- In Multi-layer perceptron, computations are difficult and time-consuming.
- In multi-layer Perceptron, it is difficult to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of the training.

Perceptron Function

Perceptron function " $f(x)$ " can be achieved as output by multiplying the input ' x ' with the learned weight coefficient ' w '.

Mathematically, we can express it as follows:

$f(x)=1$; if $w.x+b>0$

Otherwise, $f(x) =0$

- 'w' represents real-valued weights vector
- 'b' represents the bias
- 'x' represents a vector of input x values.

Characteristics of Perceptron

The perceptron model has the following characteristics.

1. Perceptron is a machine learning algorithm for supervised learning of binary classifiers.
2. In Perceptron, the weight coefficient is automatically learned.
3. Initially, weights are multiplied with input features, and the decision is made whether the neuron is fired or not.
4. The activation function applies a step rule to check whether the weight function is greater than zero.
5. The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
6. If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

Limitations of Perceptron Model

A perceptron model has limitations as follows:

- The output of a perceptron can only be a binary number (0 or 1) due to the hard limit transfer function.
- Perceptron can only be used to classify the linearly separable sets of input vectors. If input vectors are non-linear, it is not easy to classify them properly.

GENERATIVE LEARNING ALGORITHMS:

Generative approaches **try to build a model** of the *positives* and a model of the *negatives*. You can think of a model as a “*blueprint*” for a class. A decision boundary is formed where one model becomes more likely. As these create models of each class they can be used for generation.

To create these models, a generative learning algorithm learns the **joint probability** distribution $P(x, y)$.

Now time for some maths!

The joint probability can be written as:

$$P(x, y) = P(x | y) \cdot P(y) \quad \dots(i)$$

Also, using Bayes' Rule we can write:

$$P(y | x) = P(x | y) \cdot P(y) / P(x) \quad \dots(ii)$$

Since, to predict a class label y , we are only interested in the **arg max**, the denominator can be removed from (ii).

Hence to predict the label y from the training example x , generative models evaluate:

$$f(x) = \operatorname{argmax}_y P(y | x) = \operatorname{argmax}_y P(x | y) \cdot P(y)$$

The most important part in the above is $P(x | y)$. This is what allows the model to be *generative*! $P(x | y)$ means – what x (*features*) are there given *class* y . Hence, with the joint probability distribution function (i), given a y , you can calculate (“*generate*”) its corresponding x . **For this reason they are called generative models!**

Generative learning algorithms **make strong assumptions** on the data. To explain this let's look at a generative learning algorithm called Gaussian Discriminant Analysis (GDA)

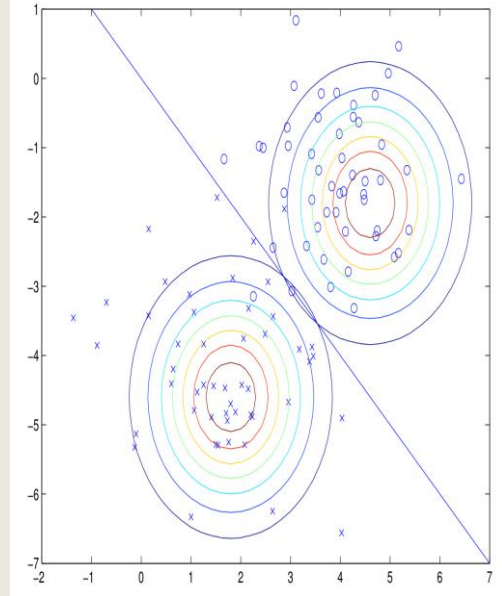
GAUSSIAN DISCRIMINANT ANALYSIS (GDA):

This model assumes that $P(x|y)$ is distributed according to a **multivariate normal distribution**.

I won't go into the maths involved but just note that the multivariate normal distribution in n -dimensions, also called the multi-variate Gaussian distribution, is parameterized by a **mean vector** $\mu \in \mathbb{R}^n$ and a **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$.

A Gaussian distribution is fit for each class. This allows us to find $P(y)$ and $P(x | y)$. Using this two we can finally find out $P(y | x)$, which is required for prediction.

For a two class dataset, pictorially what the algorithm is doing can be seen as follows:



Shown in the figure are the training set, as well as the contours of the two Gaussian distributions that have been fit to the data for each of the two classes. Also shown in the figure is the straight line giving the decision boundary at which $p(y = 1|x) = 0.5$. On one side of the boundary, we'll predict $y = 1$ to be the most likely outcome, and on the other side, we'll predict $y = 0$.

As we now have the Gaussian distribution (*model*) for each class, we can also generate new samples of the classes. The features, $\mathbf{x}$, for these new samples will be taken from the respective Gaussian distribution (*model*).

Naïve Bayes Classifier Algorithm

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.**

- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles.**

Why is it called Naïve Bayes?

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of [Bayes' Theorem](#).

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

Where,

$P(A|B)$ is Posterior probability: Probability of hypothesis A on the observed event B.

$P(B|A)$ is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

$P(A)$ is Prior Probability: Probability of hypothesis before observing the evidence.

$P(B)$ is Marginal Probability: Probability of Evidence.

Working of Naïve Bayes' Classifier:

Working of Naïve Bayes' Classifier can be understood with the help of the below example:

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions. So to solve this problem, we need to follow the below steps:

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

Problem: If the weather is sunny, then the Player should play or not?

Solution: To solve this, first consider the below dataset:

Outlook		Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes

8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	$5/14 = 0.35$
Rainy	2	2	$4/14 = 0.29$
Sunny	2	3	$5/14 = 0.35$

All	4/14=0.29	10/14=0.71	
-----	-----------	------------	--

Applying Bayes' theorem:

$$P(\text{Yes}|\text{Sunny}) = P(\text{Sunny}|\text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{Yes}) = 3/10 = 0.3$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Yes}) = 0.71$$

$$\text{So } P(\text{Yes}|\text{Sunny}) = 0.3 * 0.71 / 0.35 = \mathbf{0.60}$$

$$P(\text{No}|\text{Sunny}) = P(\text{Sunny}|\text{No}) * P(\text{No}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{NO}) = 2/4 = 0.5$$

$$P(\text{No}) = 0.29$$

$$P(\text{Sunny}) = 0.35$$

$$\text{So } P(\text{No}|\text{Sunny}) = 0.5 * 0.29 / 0.35 = \mathbf{0.41}$$

So as we can see from the above calculation that **$P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$**

Hence on a Sunny day, Player can play the game.

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for **text classification problems**.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Applications of Naïve Bayes Classifier:

- It is used for **Credit Scoring**.
- It is used in **medical data classification**.
- It can be used in **real-time predictions** because Naïve Bayes Classifier is an eager learner.
- It is used in Text classification such as **Spam filtering** and **Sentiment analysis**.

Types of Naïve Bayes Model:

There are three types of Naive Bayes Model, which are given below:

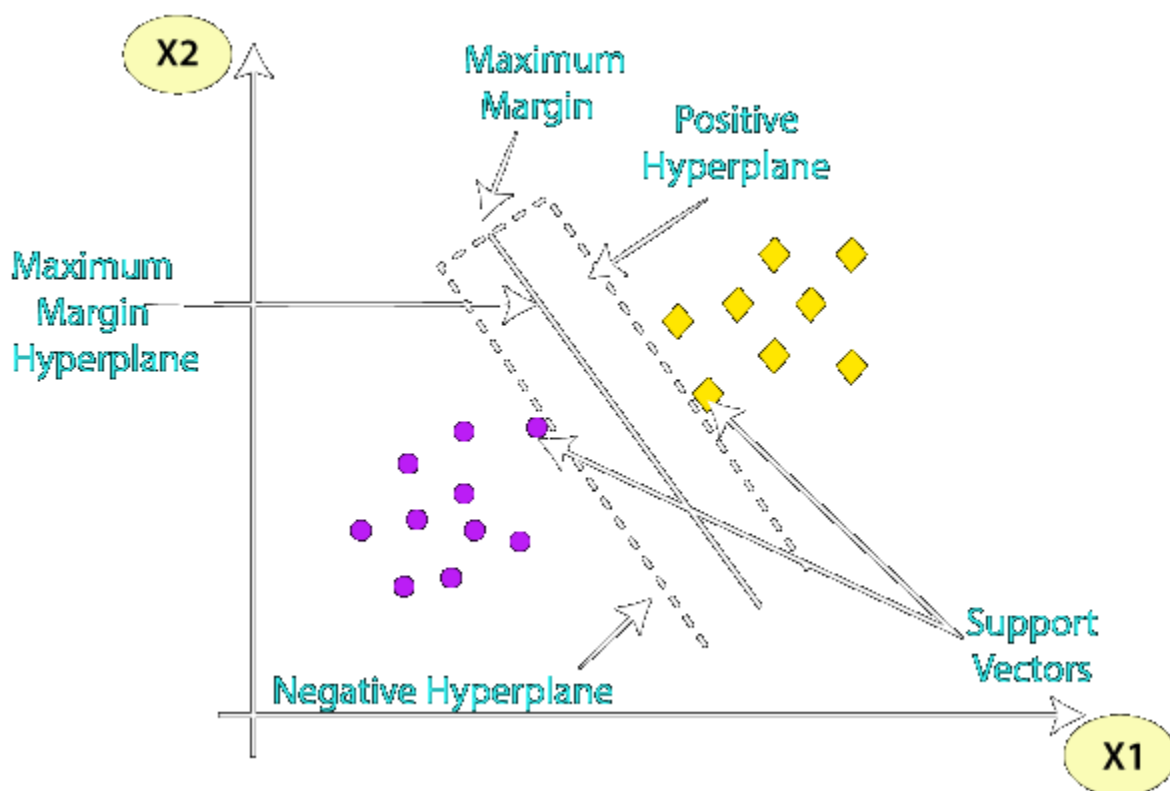
- **Gaussian:** The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial:** The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.
- **Bernoulli:** The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

Support Vector Machine Algorithm

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

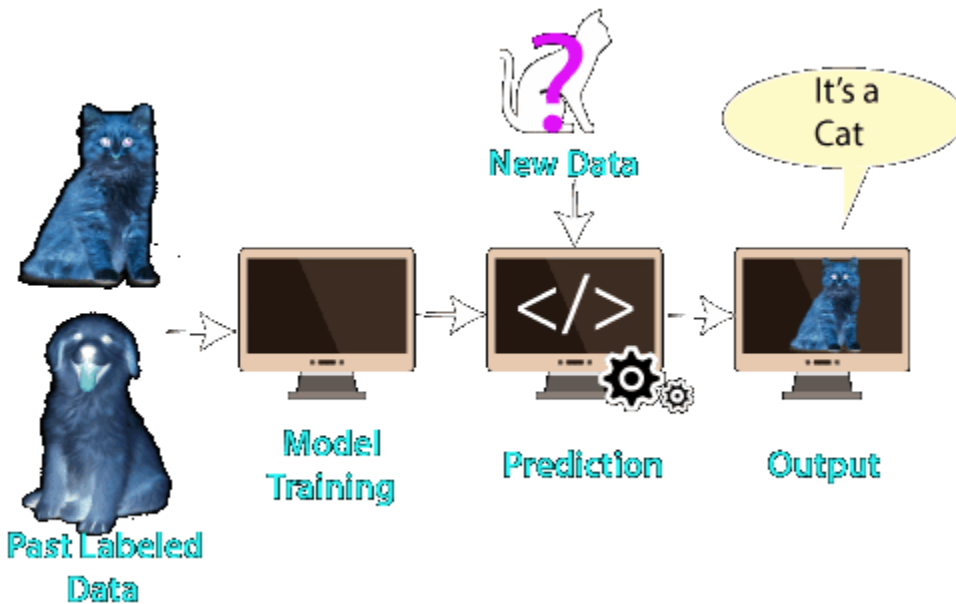
The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want

a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



SVM algorithm can be used for **Face detection, image classification, text categorization**, etc.

Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n -dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

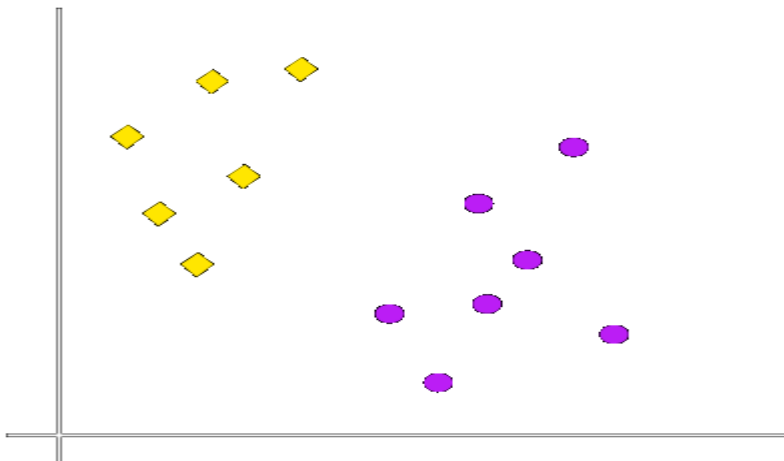
Support Vectors:

The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

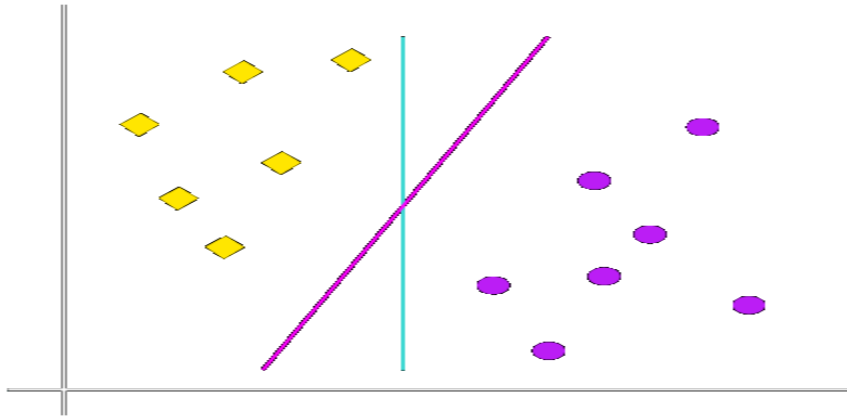
How does SVM works?

Linear SVM:

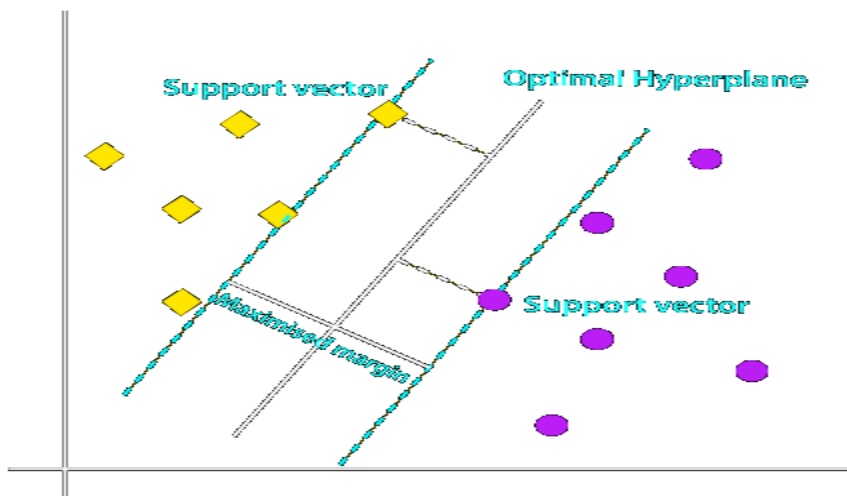
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

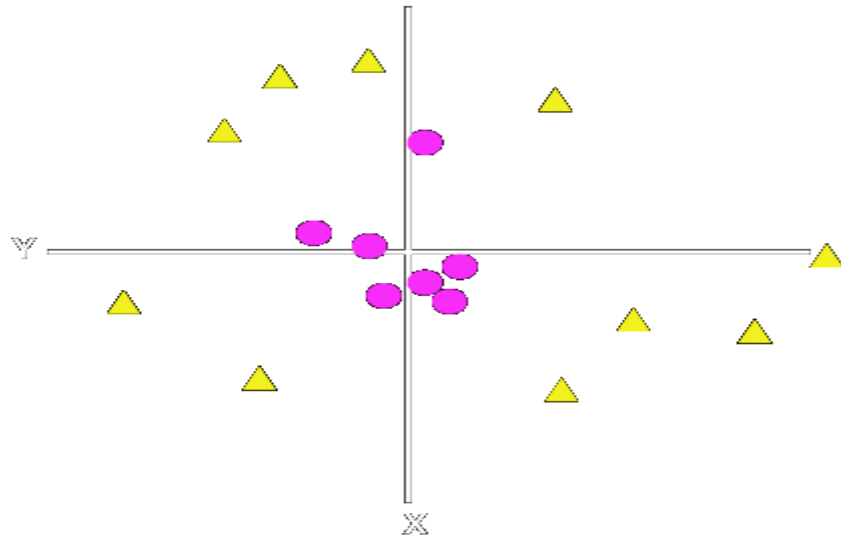


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Non-Linear SVM:

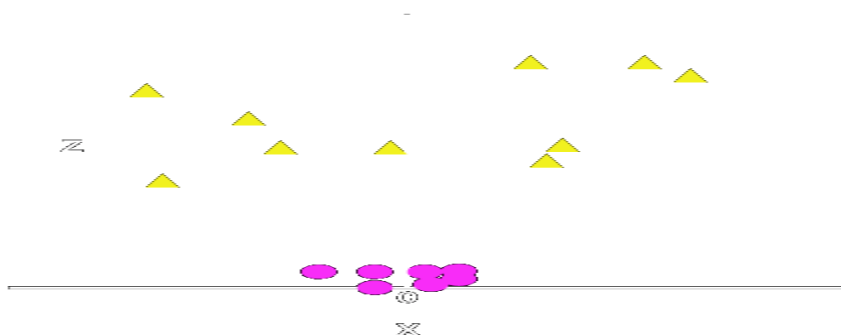
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



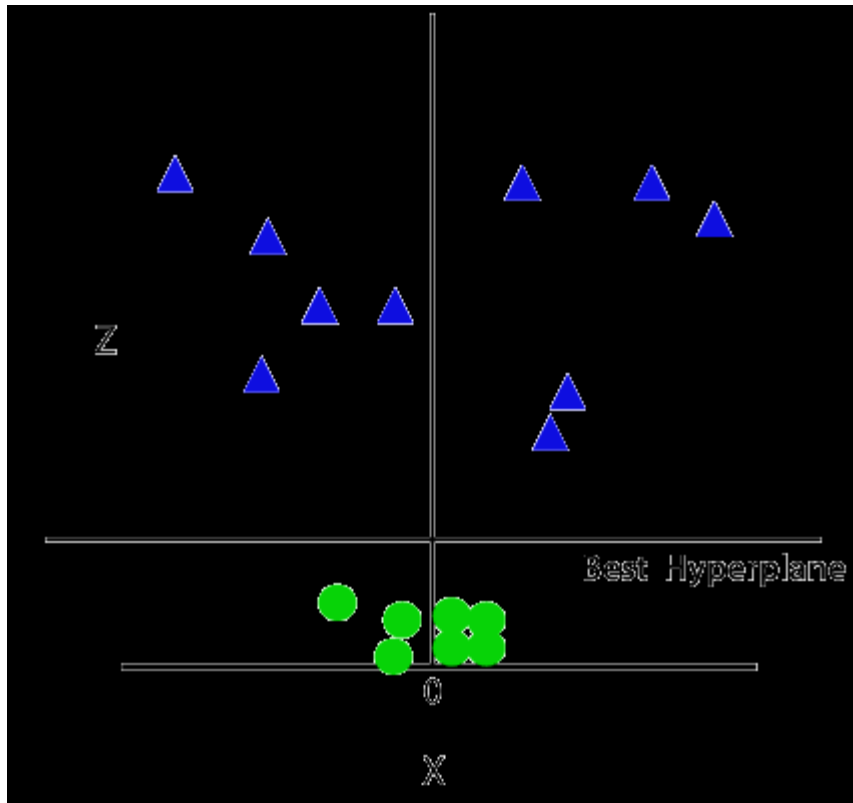
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y , so for non-linear data, we will add a third dimension z . It can be calculated as:

$$z = x^2 + y^2$$

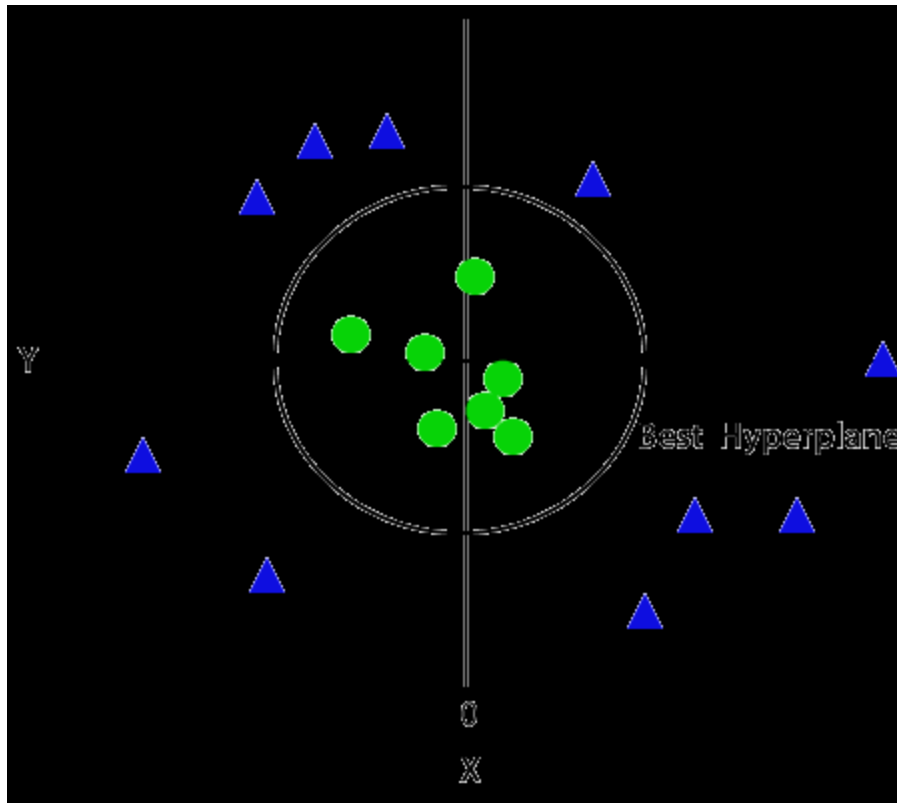
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x -axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Optimal Hyper Plane

In a binary classification problem, given a linearly separable data set, the optimal separating hyperplane is the one that correctly classifies all the data while being farthest away from the data points. In this respect, it is said to be the hyperplane that maximizes the **margin**, defined as the distance from the hyperplane to the closest data point.

The idea behind the optimality of this classifier can be illustrated as follows. New test points are drawn according to the same distribution as the training data. Thus, if the separating hyperplane is far away from the data points, previously unseen test points will most likely fall far away from the hyperplane or in the margin. As a consequence, the larger the margin is, the less likely the points are to fall on the wrong side of the hyperplane.

Finding the optimal separating hyperplane can be formulated as a convex quadratic programming problem, which can be solved with well-known techniques.

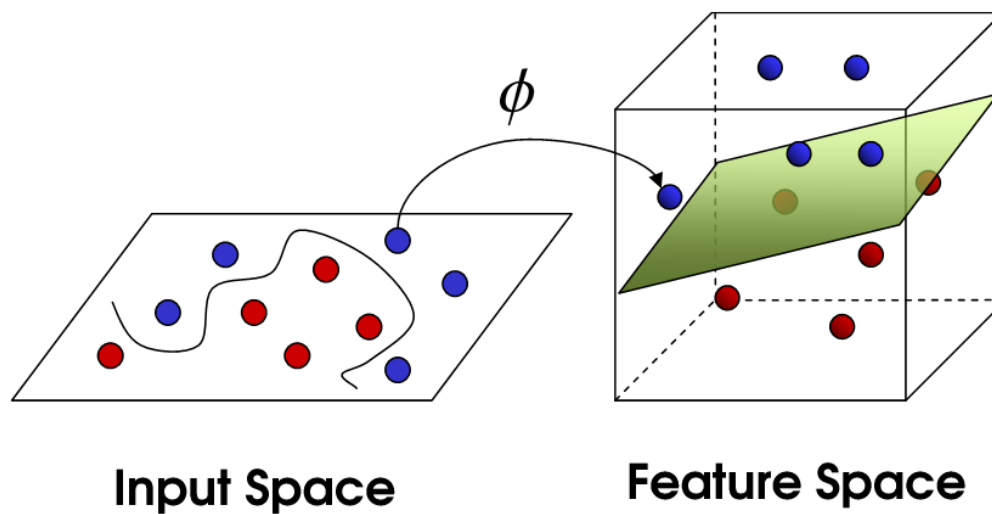
The optimal separating hyperplane should not be confused with the optimal classifier known as the Bayes classifier: the Bayes classifier is the best classifier for a given problem, independently of the available data but *unattainable in practice*, whereas the optimal separating hyperplane is only the best *linear* classifier one can produce *given a particular data set*.

The optimal separating hyperplane is one of the core ideas behind the support vector machines. In particular, it gives rise to the so-called **support vectors** which are the data points lying on the margin boundary of the hyperplane. These points support the hyperplane in the sense that they contain all the required information to compute the hyperplane: removing other points does not change the optimal separating hyperplane. Elaborating on this fact, one can actually add points to the data set without influencing the hyperplane, as long as these points lie outside of the margin.

The concept of a kernel in machine learning offers a compelling and intuitive way to understand this powerful tool used in Support Vector Machines (SVMs). At its most fundamental level, a kernel is a relatively straightforward function that operates on two vectors from the input space, commonly referred to as the X space. The primary role of this function is to return a scalar value, but the fascinating aspect of this process lies in what this scalar represents and how it is computed.

This scalar is, in essence, the dot product of the two input vectors. However, it's not computed in the original space of these vectors. Instead, it's as if this dot product is calculated in a much higher-dimensional space, known as the Z space. This is where the kernel's true power and elegance come into play. It manages to convey how close or similar these two vectors are in the Z space without the computational overhead of actually mapping the vectors to this higher-dimensional space and calculating their dot product there.

The kernel thus serves as a kind of guardian of the Z space. It allows you to glean the necessary information about the vectors in this more complex space without having to access the space directly. This approach is particularly useful in SVMs, where understanding the relationship and position of vectors in a higher-dimensional space is crucial for classification tasks.



Feature Selection Techniques in Machine Learning

Feature selection is a way of selecting the subset of the most relevant features from the original features set by removing the redundant, irrelevant, or noisy features.

A feature is an attribute that has an impact on a problem or is useful for the problem, and choosing the important features for the model is known as feature selection. Each machine learning process depends on feature engineering, which mainly contains two processes; which are Feature Selection and Feature Extraction. Although feature selection and extraction processes may have the same objective, both are completely different from each other. The main difference between them is that feature selection is about selecting the subset of the original feature set, whereas feature extraction creates new features.

Feature selection is a way of reducing the input variable for the model by using only relevant data in order to reduce overfitting in the model.

So, we can define feature Selection as, "***It is a process of automatically or manually selecting the subset of most appropriate and relevant features to be used in model building.***" Feature selection is performed by either including the important features or excluding the irrelevant features in the dataset without changing them.

Need for Feature Selection

Before implementing any technique, it is really important to understand, need for the technique and so for the Feature Selection. As we know, in machine learning, it is necessary to provide a pre-processed and good input dataset in order to get better outcomes. We collect a huge amount of data to train our model and help it to learn better. Generally, the dataset consists of noisy data, irrelevant data, and some part of useful data. Moreover, the huge amount of data also slows down the training process of the model, and with noise and irrelevant data, the model may not predict and perform well. So, it is very necessary to remove such noises and less-important data from the dataset and to do this, and Feature selection techniques are used.

Selecting the best features helps the model to perform well. For example, Suppose we want to create a model that automatically decides which car should be crushed for a spare part, and to do this, we have a dataset. This dataset contains a Model of the car, Year, Owner's name, Miles. So, in this dataset, the name of the owner does not contribute to the model performance as it does not decide if the car should be crushed or not, so we can remove this column and select the rest of the features(column) for the model building.

Below are some benefits of using feature selection in machine learning:

- **It helps in avoiding the curse of dimensionality.**
- **It helps in the simplification of the model so that it can be easily interpreted by the researchers.**
- **It reduces the training time.**
- **It reduces overfitting hence enhance the generalization.**

Feature Selection Techniques

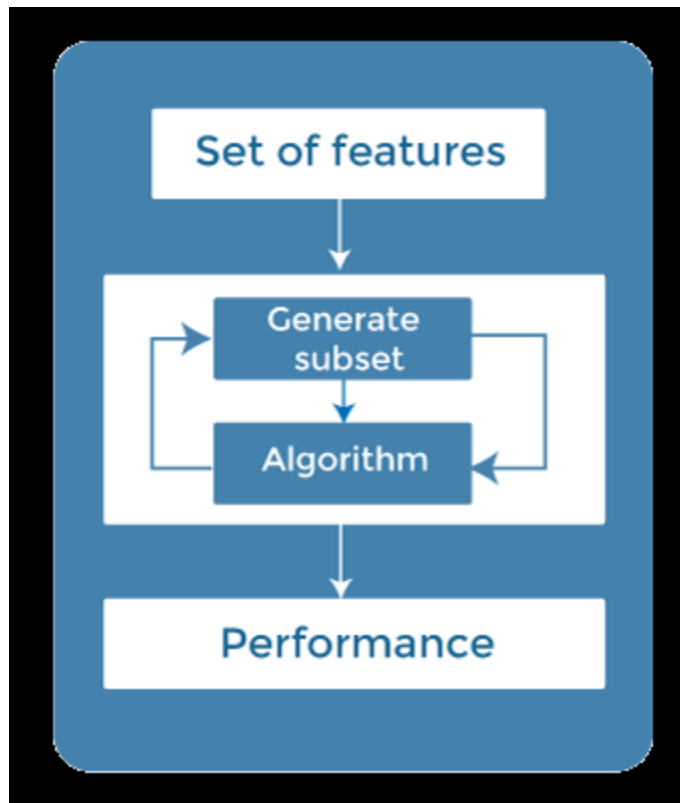
There are mainly two types of Feature Selection techniques, which are:

- **Supervised Feature Selection technique**
Supervised Feature selection techniques consider the target variable and can be used for the labelled dataset.
- **Unsupervised Feature Selection technique**
Unsupervised Feature selection techniques ignore the target variable and can be used for the unlabelled dataset.

There are mainly three techniques under supervised feature Selection:

1. Wrapper Methods

In wrapper methodology, selection of features is done by considering it as a search problem, in which different combinations are made, evaluated, and compared with other combinations. It trains the algorithm by using the subset of features iteratively.



On the basis of the output of the model, features are added or subtracted, and with this feature set, the model has trained again.

Some techniques of wrapper methods are:

- **Forward selection** - Forward selection is an iterative process, which begins with an empty set of features. After each iteration, it keeps adding on a feature and evaluates the performance to check whether it is improving the performance or not. The process continues until the addition of a new variable/feature does not improve the performance of the model.
- **Backward elimination** - Backward elimination is also an iterative approach, but it is the opposite of forward selection. This technique begins the process by considering all the features and removes the least significant feature. This elimination process continues until removing the features does not improve the performance of the model.
- **Exhaustive Feature Selection**- Exhaustive feature selection is one of the best feature selection methods, which evaluates each feature set as brute-force. It means this method tries & make each possible combination of features and return the best performing feature set.

- **Recursive Feature Elimination-**

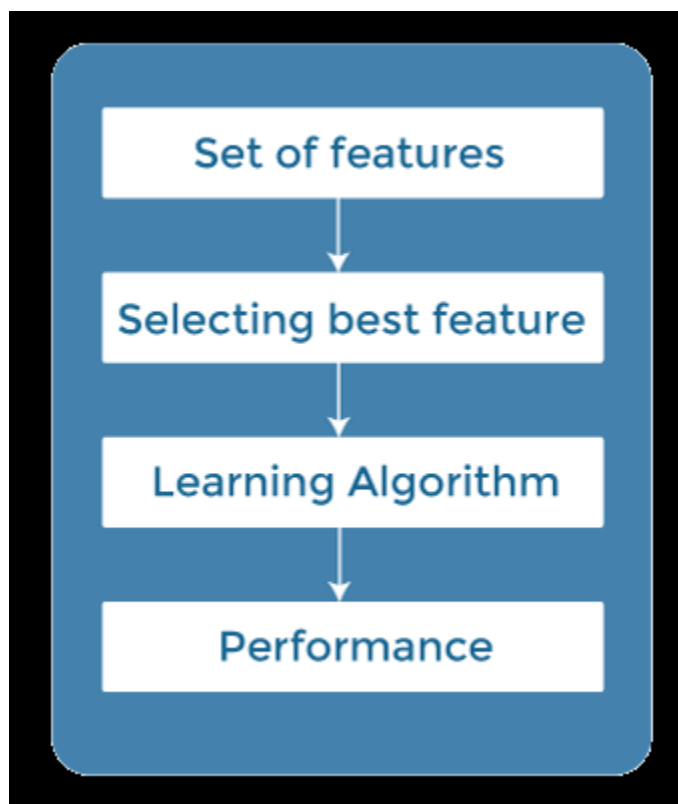
Recursive feature elimination is a recursive greedy optimization approach, where features are selected by recursively taking a smaller and smaller subset of features. Now, an estimator is trained with each set of features, and the importance of each feature is determined using *coef_attribute* or through a *feature_importances_attribute*.

2. Filter Methods

In Filter Method, features are selected on the basis of statistics measures. This method does not depend on the learning algorithm and chooses the features as a pre-processing step.

The filter method filters out the irrelevant feature and redundant columns from the model by using different metrics through ranking.

The advantage of using filter methods is that it needs low computational time and does not overfit the data.



Some common techniques of Filter methods are as follows:

- Information Gain
- Chi-square Test
- Fisher's Score
- Missing Value Ratio

Information Gain: Information gain determines the reduction in entropy while transforming the dataset. It can be used as a feature selection technique by calculating the information gain of each variable with respect to the target variable.

Chi-square Test: Chi-square test is a technique to determine the relationship between the categorical variables. The chi-square value is calculated between each feature and the target variable, and the desired number of features with the best chi-square value is selected.

Fisher's Score:

Fisher's score is one of the popular supervised technique of features selection. It returns the rank of the variable on the fisher's criteria in descending order. Then we can select the variables with a large fisher's score.

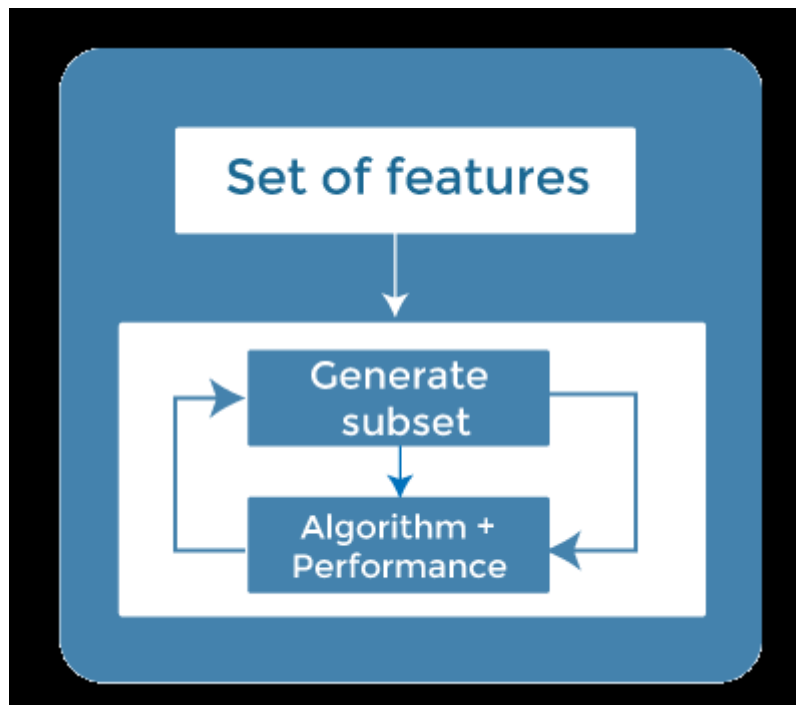
Missing Value Ratio:

The value of the missing value ratio can be used for evaluating the feature set against the threshold value. The formula for obtaining the missing value ratio is the number of missing values in each column divided by the total number of observations. The variable is having more than the threshold value can be dropped.

$$\text{Missing Value Ratio} = \frac{\text{Number of Missing values} \times 100}{\text{Total number of observations}}$$

3. Embedded Methods

Embedded methods combined the advantages of both filter and wrapper methods by considering the interaction of features along with low computational cost. These are fast processing methods similar to the filter method but more accurate than the filter method.

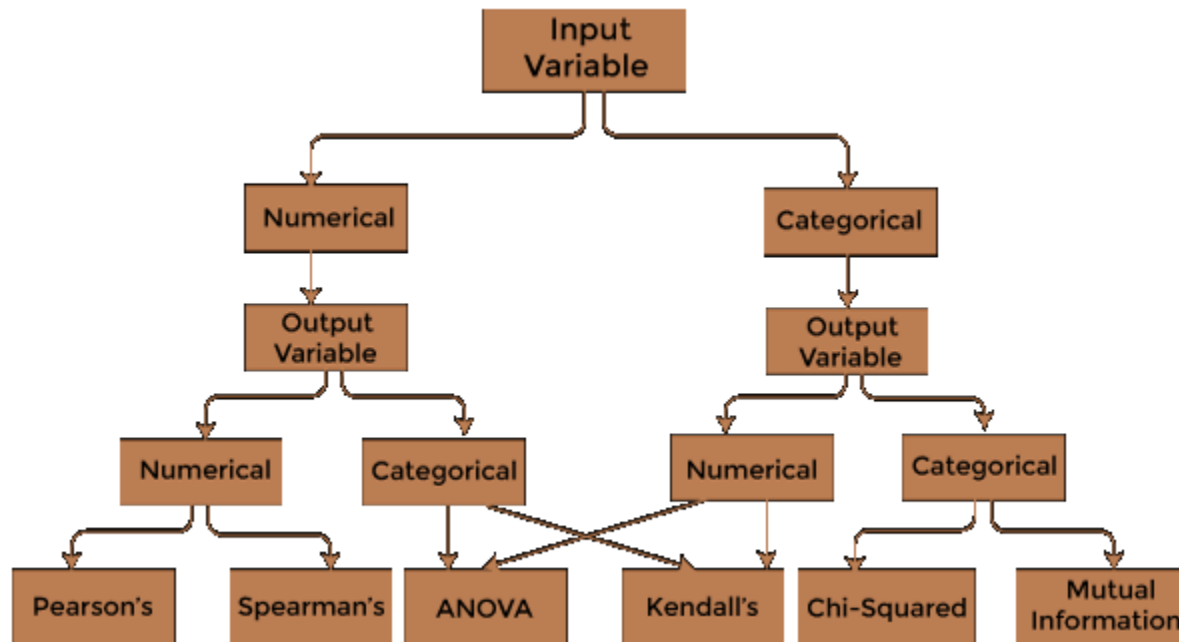


These methods are also iterative, which evaluates each iteration, and optimally finds the most important features that contribute the most to training in a particular iteration. Some techniques of embedded methods are:

- **Regularization**- Regularization adds a penalty term to different parameters of the machine learning model for avoiding overfitting in the model. This penalty term is added to the coefficients; hence it shrinks some coefficients to zero. Those features with zero coefficients can be removed from the dataset. The types of regularization techniques are L1 Regularization (Lasso Regularization) or Elastic Nets (L1 and L2 regularization).
- **Random Forest Importance** - Different tree-based methods of feature selection help us with feature importance to provide a way of selecting features. Here, feature importance specifies which feature has more importance in model building or has a great impact on the target variable. Random Forest is such a tree-based method, which is a type of bagging algorithm that aggregates a different number of decision trees. It automatically ranks the nodes by their performance or decrease in the impurity (Gini impurity) over all the trees. Nodes are arranged as per the impurity values, and thus it allows to pruning of trees below a specific node. The remaining nodes create a subset of the most important features.

How to choose a Feature Selection Method?

For machine learning engineers, it is very important to understand that which feature selection method will work properly for their model. The more we know the datatypes of variables, the easier it is to choose the appropriate statistical measure for feature selection.



To know this, we need to first identify the type of input and output variables. In machine learning, variables are of mainly two types:

- **Numerical Variables:** Variable with continuous values such as integer, float
- **Categorical Variables:** Variables with categorical values such as Boolean, ordinal, nominals.

Below are some univariate statistical measures, which can be used for filter-based feature selection:

1. Numerical Input, Numerical Output:

Numerical Input variables are used for predictive regression modelling. The common method to be used for such a case is the Correlation coefficient.

- Pearson's correlation coefficient (For linear Correlation).
- Spearman's rank coefficient (for non-linear correlation).

2. Numerical Input, Categorical Output:

Numerical Input with categorical output is the case for classification predictive modelling problems. In this case, also, correlation-based techniques should be used, but with categorical output.

- **ANOVA correlation coefficient (linear).**
- **Kendall's rank coefficient (nonlinear).**

3. Categorical Input, Numerical Output:

This is the case of regression predictive modelling with categorical input. It is a different example of a regression problem. We can use the same measures as discussed in the above case but in reverse order.

4. Categorical Input, Categorical Output:

This is a case of classification predictive modelling with categorical Input variables.

The commonly used technique for such a case is Chi-Squared Test. We can also use Information gain in this case.

Model selection is an important machine learning process that focuses on adopting the best suitable algorithm and model for a specific dataset. It includes assessing and comparing different models to identify the one that produces the best results. Different features are considered, and varying metrics are used to reach a conclusion.

Types of machine learning models

Machine learning models are sorted and categorized under different types to make the model selection easier and more accurate. We can verify our requirements and scenario with how each type processes and then choose the models from that category.

Here are the major types under which the models are categorized based on their behavior.

Type	What does it do	Examples
------	-----------------	----------

Classification	Predicts categorical variables for a given dataset.	Decision trees, logistic regression, neural networks.
Regression	Predicts a continuous value for a given input.	Polynomial regression, vector regression, linear regression.
Clustering	Uses unsupervised learning algorithms to group similar data points.	Hierarchical clustering, K-means DBSCAN
Dimensions reduction	Reduces the number of features in a dataset.	Linear discriminant analysis, t-SNE principal component analysis
Generative	Generates new data that is comparable to the training dataset.	Autoregressive models, generative adversarial networks, variational autoencoders.

Features to consider

Selecting a suitable model is the most crucial step in machine learning because it influences the observations and the results obtained. Let's discuss a few important features when selecting a model.

Complexity

Determine the complexity of the problem that is to be solved. There might be some cases where simple models are sufficient enough to solve the issues, but at times there is a necessity to use

complex models. Hence, the size of the dataset, the complexity of the inputs, and potential connections should be kept under consideration when selecting the model.

Data availability

Analyze the existing data accessibility and quality. If the dataset is limited, it is preferred to use simpler models with limited parameters than a complicated model with many parameters to avoid overfitting. It is essential to consider the missing data, outliers, noise, and models' responses to the difficulties before selecting the model.

Regulations

Analyze the model's capacity to determine whether it fits well on the fresh and untested data. We can incorporate penalty terms into the model's objective function and implement approaches such as L1 or L2 regularization to overcome the overfitting issues. The regularised models potentially perform better on sparse training data.

Domain Expertise

Consider your expertise and domain knowledge. On the basis of previous knowledge of the data or particular features of the domain, consider if particular models are appropriate for the task. Models that are more likely to capture important patterns can be found by using domain expertise to direct the selection process.

Resource constraints

Take into account any resource limitations you may have, such as constrained memory space, processing speed, or time. Make that the chosen model can be successfully implemented using the resources at hand. Some models require significant resources during training or inference.

Scalability

If you're working with massive datasets or real-time applications, take the model's scalability and computing efficiency into consideration. Deep neural networks and support vector machines are two examples of models that could need a lot of time and computing power to train.

Interpretability

Consider whether the model's interpretability is crucial in your particular setting. Some models, like decision trees or linear regression, offer interpretability by giving precise insights into the correlations between the input data and the desired outcome. Complex models, such as neural networks, may perform better but offer less interpretability.

Steps for model selection

When finding the best suitable model, we identify the dataset and define the aim and purpose of acquiring a machine learning model. Once it is done, we follow a simple chronological order to reach the best option among all the available models.

Standard model selection steps.

- **Formulate problem:** Precisely define the problem to be catered to, predictions to be made, and the expected task it should perform.
- **Choose potential models:** Choose models that are appropriate for the requirements. The chosen models can be simple, like decision trees and linear regression, or complex, like deep neural networks and random forests.
- **Do hyperparameter tuning:** Find the best combination of hyperparameters for the model, like learning rate and regularisation strength, to achieve optimal performance. They help to avoid overfitting and overfitting and underfitting.
- **Train and evaluate each model:** Train each model using a subset of the original dataset, and measure its performance using the other subset that is not trained to evaluate its effectiveness.
- **Compare the performance and accuracy:** Compare the performance of the chosen models based on different metrics, including the F1-score, mean squared error, accuracy, precision, and recall. Also, consider factors like data handling capabilities, interpretability, and computational difficulty.
- **Finalize the best-suited model:** Based on the observation and comparison results, select the model that performs the best. The finalized model can be used on the fresh dataset to perform the required tasks and make predictions.

Combining classifiers

Ensemble learning in [machine learning](#) combines multiple individual models to create a stronger, more accurate predictive model. By leveraging the diverse strengths of different models, ensemble learning aims to mitigate errors, enhance performance, and increase the overall robustness of predictions, leading to improved results across various tasks in machine learning and data analysis.

What is Bagging?

Bagging (Bootstrap Aggregating) is an ensemble learning technique designed to improve the accuracy and stability of [machine learning algorithms](#). It involves the following steps:

1. **Data Sampling:** Creating multiple subsets of the training dataset using bootstrap sampling (random sampling with replacement).
2. **Model Training:** Training a separate model on each subset of the data.
3. **Aggregation:** Combining the predictions from all individual models (averaged for regression or majority voting for classification) to produce the final output.

Key Benefits:

- **Reduces Variance:** By averaging multiple predictions, bagging reduces the variance of the model and helps prevent overfitting.
- **Improves Accuracy:** Combining multiple models usually leads to better performance than individual models.

Example of Bagging Algorithms:

- [Random Forests](#) (an extension of bagging applied to decision trees)

What is Boosting?

Boosting is another ensemble learning technique that focuses on creating a strong model by combining several weak models. It involves the following steps:

1. **Sequential Training:** Training models sequentially, each one trying to correct the errors made by the previous models.
2. **Weight Adjustment:** Each instance in the training set is weighted. Initially, all instances have equal weights. After each model is trained, the weights of

misclassified instances are increased so that the next model focuses more on difficult cases.

3. **Model Combination:** Combining the predictions from all models to produce the final output, typically by weighted voting or weighted averaging.

Key Benefits:

- **Reduces Bias:** By focusing on hard-to-classify instances, boosting reduces bias and improves the overall model accuracy.
- **Produces Strong Predictors:** Combining weak learners leads to a strong [predictive model](#).

Example of Boosting Algorithms:

- [AdaBoost](#)
- [Gradient Boosting Machines \(GBM\)](#)
- [XGBoost](#)
- [LightGBM](#)

Similarities between Bagging and Boosting

Bagging (Bootstrap Aggregating) and Boosting are both ensemble learning techniques designed to improve the performance of machine learning models by combining the predictions of multiple base models.

Both bagging and boosting involve training multiple models on different subsets of the training data and then combining their predictions to make a final prediction. These techniques aim to reduce the variance of the model and improve its overall accuracy and stability.

Additionally, using bagging and boosting with various base models, such as decision trees, to create a diverse set of models that capture different aspects of the data.

Differences between Bagging and Boosting

While bagging and boosting share some similarities, their approach and methodology differ.

Bagging trains each base model independently and in parallel, using bootstrap sampling to create multiple subsets of the training data. The final prediction is then made by averaging the predictions of all base models. Bagging focuses on reducing variance and overfitting by creating diverse models.

In contrast, boosting trains models sequentially, with each subsequent model focusing on correcting the errors made by the previous ones. Boosting adjusts the weights of training instances to prioritize difficult-to-classify instances, thus reducing bias and improving predictive accuracy. The final prediction is made by combining the predictions of all models, typically using a weighted voting or averaging approach.

Additionally, while bagging is relatively simple and easy to parallelize, boosting is more complex due to its sequential nature and may be more prone to overfitting if not properly controlled.

Ensemble learning in [machine learning](#) combines multiple individual models to create a stronger, more accurate predictive model. By leveraging the diverse strengths of different models, ensemble learning aims to mitigate errors, enhance performance, and increase the overall robustness of predictions, leading to improved results across various tasks in machine learning and data analysis.

What is Bagging?

Bagging (Bootstrap Aggregating) is an ensemble learning technique designed to improve the accuracy and stability of [machine learning algorithms](#). It involves the following steps:

1. **Data Sampling:** Creating multiple subsets of the training dataset using bootstrap sampling (random sampling with replacement).
2. **Model Training:** Training a separate model on each subset of the data.
3. **Aggregation:** Combining the predictions from all individual models (averaged for regression or majority voting for classification) to produce the final output.

Key Benefits:

- **Reduces Variance:** By averaging multiple predictions, bagging reduces the variance of the model and helps prevent overfitting.

- **Improves Accuracy:** Combining multiple models usually leads to better performance than individual models.

Example of Bagging Algorithms:

- [Random Forests](#) (an extension of bagging applied to decision trees)

Also Read: [Beginner's Guide to Ensemble Learning in Python](#)

What is Boosting?

Boosting is another ensemble learning technique that focuses on creating a strong model by combining several weak models. It involves the following steps:

1. **Sequential Training:** Training models sequentially, each one trying to correct the errors made by the previous models.
2. **Weight Adjustment:** Each instance in the training set is weighted. Initially, all instances have equal weights. After each model is trained, the weights of misclassified instances are increased so that the next model focuses more on difficult cases.
3. **Model Combination:** Combining the predictions from all models to produce the final output, typically by weighted voting or weighted averaging.

Key Benefits:

- **Reduces Bias:** By focusing on hard-to-classify instances, boosting reduces bias and improves the overall model accuracy.

- **Produces Strong Predictors:** Combining weak learners leads to a strong [predictive model](#).

Example of Boosting Algorithms:

- [AdaBoost](#)
- [Gradient Boosting Machines \(GBM\)](#)
- [XGBoost](#)
- [LightGBM](#)

Similarities between Bagging and Boosting

Bagging (Bootstrap Aggregating) and Boosting are both ensemble learning techniques designed to improve the performance of machine learning models by combining the predictions of multiple base models.

Both bagging and boosting involve training multiple models on different subsets of the training data and then combining their predictions to make a final prediction.

These techniques aim to reduce the variance of the model and improve its overall accuracy and stability.

Additionally, using bagging and boosting with various base models, such as decision trees, to create a diverse set of models that capture different aspects of the data.

Differences between Bagging and Boosting

While bagging and boosting share some similarities, their approach and methodology differ.

Bagging trains each base model independently and in parallel, using bootstrap sampling to create multiple subsets of the training data. The final prediction is then made by averaging the predictions of all base models. Bagging focuses on reducing variance and overfitting by creating diverse models.

In contrast, boosting trains models sequentially, with each subsequent model focusing on correcting the errors made by the previous ones. Boosting adjusts the weights of training instances to prioritize difficult-to-classify instances, thus reducing bias and improving predictive accuracy. The final prediction is made by combining the predictions of all models, typically using a weighted voting or averaging approach.

Additionally, while bagging is relatively simple and easy to parallelize, boosting is more complex due to its sequential nature and may be more prone to overfitting if not properly controlled.

Ada Boost algorithm in Machine Learning.

Machine learning algorithms have the notable potential to make predictions and decisions primarily based on patterns and information. However, not all algorithms are created equal. Some perform better on positive sorts of data, even as others may additionally warfare. AdaBoost, short for Adaptive Boosting, is a powerful ensemble learning algorithm that could decorate the overall Performance of susceptible, inexperienced persons and create a sturdy classifier. In this article, we're going to dive into the world of AdaBoost, exploring its principles, working mechanism, and practical applications.

Introduction to AdaBoost

AdaBoost is a boosting set of rules that was added with the aid of Yoav Freund and Robert Schapire in 1996. It is part of a class of ensemble getting-to-know strategies that aim to improve the overall performance of gadget getting-to-know fashions by combining the outputs of a couple of weaker fashions, known as vulnerable, inexperienced persons or base novices. The fundamental idea at the back of AdaBoost is to offer greater weight to the schooling instances that are misclassified through the modern-day model, thereby focusing on the samples that are tough to classify.

How AdaBoost Works

To understand how AdaBoost works, smash down its working mechanism right into a step-by-step process:

1. Weight Initialization

At the start, every schooling instance is assigned an identical weight. These weights determine the importance of every example in the getting-to-know method.

2. Model Training

A weak learner is skilled at the dataset, with the aim of minimizing class errors. A weak learner is usually an easy model, which includes a selection stump (a one-stage decision tree) or a small neural network.

3. Weighted Error Calculation

After the vulnerable learner is skilled, its miles are used to make predictions at the education dataset. The weighted mistakes are then calculated by means of summing up the weights of the misclassified times. This step emphasizes the importance of the samples which are tough to classify.

4. Model Weight Calculation

The weight of the susceptible learner is calculated primarily based on their Performance in classifying the training data. Models that perform properly are assigned higher weights, indicating that they're more reliable.

5. Update Instance Weights

The example weights are updated to offer more weight to the misclassified samples from the previous step. This adjustment focuses on the studying method at the times that the present-day model struggles with.

6. Repeat

Steps 2 through five are repeated for a predefined variety of iterations or till a distinctive overall performance threshold is met.

7. Final Model Creation

The very last sturdy model (also referred to as the ensemble) is created by means of combining the weighted outputs of all weak newcomers. Typically, the fashions with better weights have an extra influence on the final choice.

8. Classification

To make predictions on new records, AdaBoost uses the very last ensemble model. Each vulnerable learner contributes its prediction, weighted with the aid of its significance, and the blended result is used to categorize the enter.

Key Concepts in AdaBoost

To gain deeper information about AdaBoost, it's critical to be acquainted with some key principles associated with the algorithm:

1. Weak Learners

Weak novices are the individual fashions that make up the ensemble. These are generally fashions with accuracy barely higher than random hazards. In the context of AdaBoost, weak beginners are trained sequentially, with each new model focusing on the instances that preceding models determined difficult to classify.

2. Strong Classifier

The strong classifier, additionally known as the ensemble, is the final version created by combining the predictions of all weak first-year students. It has the collective know-how of all of the fashions and is capable of making correct predictions.

3. Weighted Voting

In AdaBoost, every susceptible learner contributes to the very last prediction with a weight-based totally on its Performance. This weighted vote-casting machine ensures that the greater correct fashions have a greater say in the final choice.

4. Error Rate

The error rate is the degree of ways a vulnerable learner plays on the schooling statistics. It is used to calculate the load assigned to each vulnerable learner. Models with lower error fees are given higher weights.

5. Iterations

The range of iterations or rounds in AdaBoost is a hyperparameter that determines what number of susceptible newbies are educated. Increasing the range of iterations may additionally result in a more complex ensemble; however, it can also increase the risk of overfitting.

Advantages of AdaBoost

AdaBoost gives numerous blessings that make it a popular choice in gadget mastering:

1. Improved Accuracy

AdaBoost can notably improve the accuracy of susceptible, inexperienced persons, even when the usage of easy fashions. By specializing in misclassified instances, it adapts to the tough areas of the records distribution.

2. Versatility

AdaBoost can be used with a number of base newbies, making it a flexible set of rules that may be carried out for unique forms of problems.

3. Feature Selection

It routinely selects the most informative features, lowering the need for giant function engineering.

4. Resistance to Overfitting

AdaBoost tends to be much less at risk of overfitting compared to a few different ensemble methods, thanks to its recognition of misclassified instances.

Limitations and Challenges

While AdaBoost is an effective algorithm, it is important to be aware of its barriers and challenges:

1. Sensitivity to Noisy Data

AdaBoost may be sensitive to noisy facts and outliers because it offers greater weight to misclassified times. Outliers can dominate the studying system and result in suboptimal consequences.

2. Computationally Intensive

Training AdaBoost may be computationally intensive, especially while using a massive wide variety of susceptible learners. This could make it much less appropriate for real-time applications.

3. Overfitting

Although AdaBoost is much less prone to overfitting than a few different algorithms, it may nonetheless overfit if the number of iterations is too excessive.

4. Model Selection

Selecting the proper vulnerable learner and tuning hyperparameters may be difficult, as the Performance of AdaBoost is noticeably dependent on these alternatives.

Practical Applications

AdaBoost has found applications in a huge range of domains, along with but not constrained to:

1. Face Detection

AdaBoost has been used in computer imagination and prescient for obligations like face detection, in which it allows the perception of faces in pics or motion pictures.

2. Speech Recognition

In speech popularity, AdaBoost can be used to improve the accuracy of phoneme or word popularity structures.

3. Anomaly Detection

AdaBoost can be applied to anomaly detection problems in numerous fields, such as finance, healthcare, and cybersecurity.

4. Natural Language Processing

In NLP, AdaBoost can decorate the overall Performance of sentiment analysis and textual content category fashions.

5. Biology and Bioinformatics

AdaBoost has been used for protein type, gene prediction, and different bioinformatics duties.

Implementation and Understanding

Step 1 - Creating the First Base Learner

In step one of the AdaBoost set of rules, we begin by growing the first base learner, which is basically a selection stump, and we'll call it f1. For this case, we've got three features (f1, f2, and f3) in our dataset, so we're going to create three stumps. The preference of which stump to apply is because the first base learner is based totally on the assessment of Gini impurity or entropy, just like decision timber. We select the stump with the bottom Gini impurity or entropy, and in this situation, let's anticipate that f1 has the bottom entropy.

Step 2 - Calculating the Total Error (TE)

Next, we calculate the Total Error (TE), which represents the sum of errors within the classified records for the sample weights. In this case, there may be the handiest one error, so TE is calculated as 1/5.

Step 3 - Calculating the Performance of the Stump

The Performance of the stump is calculated by the usage of the system:

$$\text{Performance} = \frac{1}{2} \ln(1 - \text{TE} / \text{TE})$$

In our case, TE is 1/5. By substituting this cost into the system and fixing it, we discover that the overall Performance of the stump is approximately 0.693.

Step 4 - Updating Weights

The subsequent step entails updating the pattern weights. For incorrectly labeled facts, the formulation for updating the weights is:

$$\text{New Sample Weight} = \text{Sample Weight} \times e^{\text{Performance}}$$

In this situation, the Sample Weight is 1/5, and the Performance is 0.693. So, the updated weight for incorrectly classified statistics is about 0.399.

For correctly labeled facts, the equal formulation is used, but with a terrible Performance fee:

$$\text{New Sample Weight} = \text{Sample Weight} \times e^{-\text{Performance}}$$

In this example, the up-to-date weight for successfully labeled statistics is about 0.100.

The sum of all of the up-to-date weights needs to be 1, preferably. However, in this example, the sum is 0.799.

To normalize the weights, each updated weight is divided by way of the full sum of the updated weights. For example, if the updated weight is 0.399 and the entire sum of up-to-date weights is 0.799, then the normalized weight is $0.399 / 0.799 \approx 0.50$. This normalization guarantees that the sum of weights turns into approximately 1.

Step 5 - Creating a New Dataset

In this step, we create a new dataset from the previous one, considering the normalized weights. The new dataset could have greater instances of incorrectly labeled information than effectively categorized ones. To create this new dataset, the algorithm divides the normalized weights into buckets. For instance, if the normalized weights range from 0 to 0.13, 0.013, to 0.63 to 0.76, and so on, the set of rules selects data randomly from those buckets primarily based on their weights.

This system is repeated in multiple instances (in this situation, 5 iterations) to form the brand-new dataset. Incorrectly classified records could likely be selected extra regularly, as their weights are better. The result is a new dataset to be used to educate the following choice tree/stump within the AdaBoost algorithms.

The AdaBoost algorithm continues iterating through these steps, sequentially deciding on stumps and creating new datasets, with a focal point on correctly classifying the data that were formerly misclassified. This iterative procedure facilitates AdaBoost to improve the overall performance of its ensemble of weak learners.

The way of Deciding the Output of the Algorithm for Test Data

1. Multiple Decision Trees or Stumps: AdaBoost creates a couple of choice bushes or stumps during schooling. These bushes are like different experts, each with their critiques on how to classify information.
2. Passing Through the Trees: When you have a new piece of facts to categorize (check data), it's like asking each of these experts for their opinion. You pass the records through every tree one after the other.
3. Individual Predictions: Each tree (or expert) makes its very own prediction for the facts. For instance, one tree might say, "I assume it's a 1," even as any other says, "I additionally assume it's a 1," and some other might say, "I think it's a 0."

4. Weighted Opinions: AdaBoost does not treat all professionals (trees) similarly. It will pay extra attention to the professionals who have been correct at making predictions in the past. It offers these accurate professionals better significance or weight.
5. Majority Decision: The very last choice is made by counting the opinions of these specialists, however, giving extra significance to the ones which have been proper more frequently. If maximum specialists agree (in this example, if the bulk of them say it is a 1), then the very last choice is to categorize the facts as 1.

So, in our example, if the first trees (stumps) say it's a 1, and the third one says it is a 0, the majority opinion wins, and the final output for the test information might be 1.

This technique of mixing the opinions of multiple experts, with extra weight given to the better professionals, is what makes AdaBoost a powerful algorithm for classification responsibilities. It leverages the strengths of each professional to make a more correct final selection.

EVALUATING LEARNING ALGORITHMS

Evaluating your machine learning algorithm is an essential part of any project. Your model may give you satisfying results when evaluated using a metric *say accuracy_score* but may give poor results when evaluated against other metrics such as *logarithmic_loss* or any other such metric. Most of the times we use classification accuracy to measure the performance of our model, however it is not enough to truly judge our model. In this post, we will cover different types of evaluation metrics available.

Classification Accuracy

Logarithmic Loss

Confusion Matrix

Area under Curve

F1 Score

Mean Absolute Error

Mean Squared Error

Here are 12 important model evaluation metrics commonly used in machine learning:

Confusion Matrix

A confusion matrix is an $N \times N$ matrix, where N is the number of predicted classes. For the problem in hand, we have $N=2$, and hence we get a 2×2 matrix. It is a performance measurement for machine learning classification problems where the output can be two or more classes. Confusion matrix is a table with 4 different combinations of predicted and actual values. It is extremely useful for measuring precision-recall, Specificity, Accuracy, and most importantly, AUC-ROC curves.

Here are a few definitions you need to remember for a confusion matrix:

- **True Positive:** You predicted positive, and it's true.
- **True Negative:** You predicted negative, and it's true.
- **False Positive: (Type 1 Error):** You predicted positive, and it's false.

- **False Negative: (Type 2 Error):** You predicted negative, and it's false.
- **Accuracy:** the proportion of the total number of correct predictions that were correct.
- **Positive Predictive Value or Precision:** the proportion of positive cases that were correctly identified.
- **Negative Predictive Value:** the proportion of negative cases that were correctly identified.
- **Sensitivity or Recall:** the proportion of actual positive cases which are correctly identified.
- **Specificity:** the proportion of actual negative cases which are correctly identified.
- **Rate:** It is a measuring factor in a confusion matrix. It has also 4 types TPR, FPR, TNR, and FNR.

Confusion Matrix		Target			
		Positive	Negative		
Model	Positive	a	b	Positive Predictive Value	$a/(a+b)$
	Negative	c	d	Negative Predictive Value	$d/(c+d)$
		Sensitivity	Specificity	Accuracy = $(a+d)/(a+b+c+d)$	
		$a/(a+c)$	$d/(b+d)$		

The accuracy for the problem in hand comes out to be 88%. As you can see from the above two tables, the Positive Predictive Value is high, but the negative predictive value is quite low. The same holds for Sensitivity and Specificity. This is primarily driven by the threshold value we have chosen. If we decrease our threshold value, the two pairs of starkly different numbers will come closer.

In general, we are concerned with one of the above-defined metrics. For instance, in a pharmaceutical company, they will be more concerned with a minimal wrong positive diagnosis. Hence, they will be more concerned about high Specificity. On the other hand, an attrition model will be more concerned with Sensitivity. Confusion matrices are generally used only with class output models.

F1 Score

In the last section, we discussed precision and recall for classification problems and also highlighted the importance of choosing a precision/recall basis for our use case. What if, for a use case, we are trying to get the best precision and recall at the same time? F1-Score is the harmonic mean of precision and recall values for a classification problem. The formula for F1-Score is as follows:

$$F_1 = \left(\frac{\text{recall}^{-1} + \text{precision}^{-1}}{2} \right)^{-1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Now, an obvious question that comes to mind is why you are taking a harmonic mean and not an arithmetic mean. This is because HM punishes extreme values more. Let us understand this with an example. We have a binary classification model with the following results:

Precision: 0, Recall: 1

Here, if we take the arithmetic mean, we get 0.5. It is clear that the above result comes from a dumb classifier that ignores the input and predicts one of the classes as output. Now, if we were to take HM, we would get 0 which is accurate as this model is useless for all purposes.

This seems simple. There are situations, however, for which a data scientist would like to give a percentage more importance/weight to either precision or recall. Altering the above expression a bit such that we can include an adjustable parameter beta for this purpose, we get:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}.$$

Fbeta measures the effectiveness of a model with respect to a user who attaches β times as much importance to recall as precision.

Gain and Lift Charts

Gain and Lift charts are mainly concerned with checking the rank ordering of the probabilities. Here are the steps to build a Lift/Gain chart:

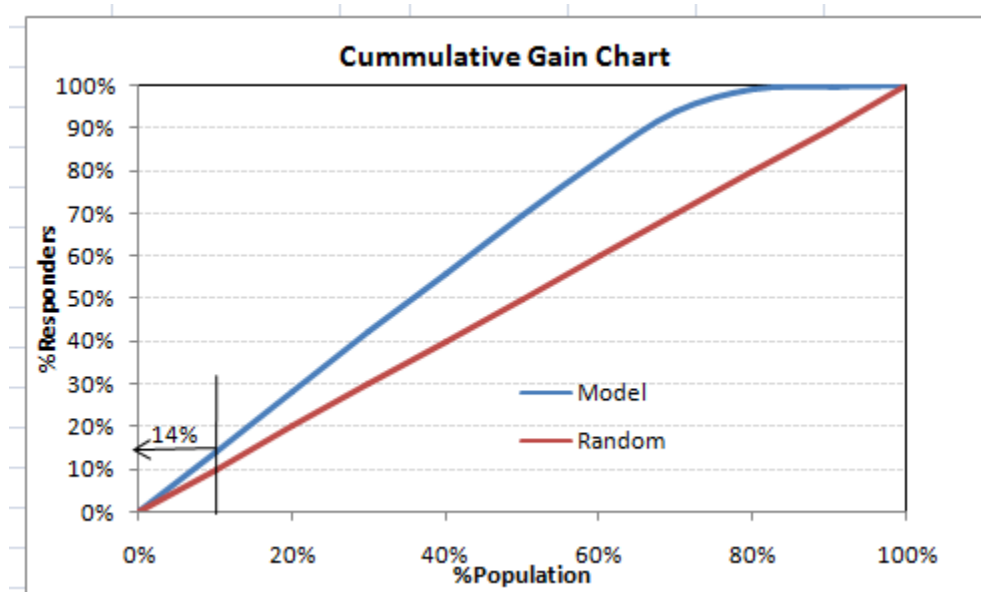
- Step 1: Calculate the probability for each observation
- Step 2: Rank these probabilities in decreasing order.
- Step 3: Build deciles with each group having almost 10% of the observations.

- Step 4: Calculate the response rate at each decile for Good (Responders), Bad (Non-responders), and total.

You will get the following table from which you need to plot Gain/Lift charts:

Lift/Gain	Column Labels			%Rights	%Wrongs	%Population	Cum %Right	Cum %Pop	Lift @decile	Total Lift
Row Labels	0	1	Grand Total	0%	0%	0%	0%	0%		
1		543	543	14%	0%	10%	14%	10%	141%	141%
2		2	544	14%	0%	10%	28%	20%	141%	141%
3		7	537	14%	0%	10%	42%	30%	139%	141%
4		15	529	14%	1%	10%	56%	40%	137%	140%
5		20	524	14%	1%	10%	69%	50%	136%	139%
6		42	502	13%	3%	10%	83%	60%	130%	138%
7		104	440	11%	7%	10%	94%	70%	114%	134%
8		345	199	5%	22%	10%	99%	80%	52%	124%
9		515	29	1%	32%	10%	100%	90%	8%	111%
10		540	5	0%	34%	10%	100%	100%	1%	100%
Grand Total	1590	3850	5440							

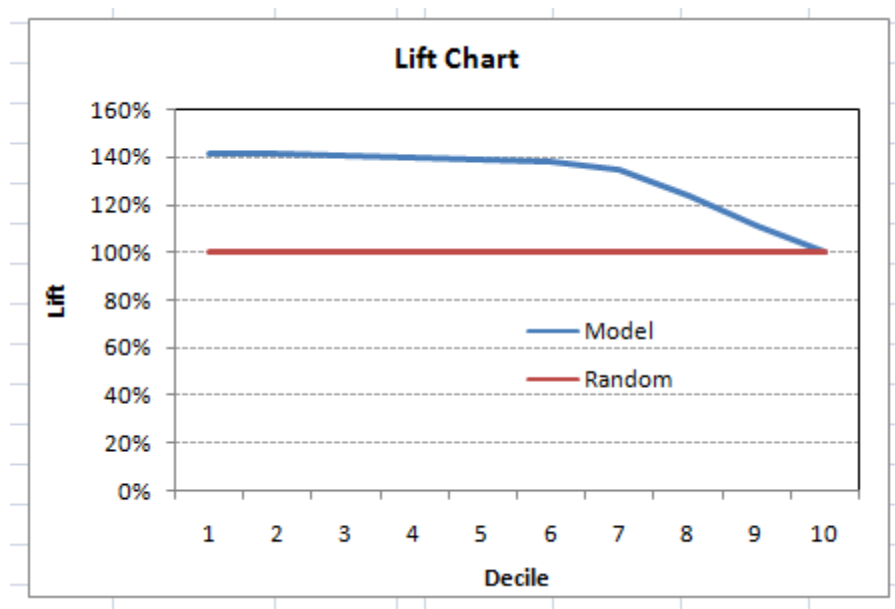
This is a very informative table. The cumulative Gain chart is the graph between Cumulative %Right and Cumulative %Population. For the case in hand, here is the graph:



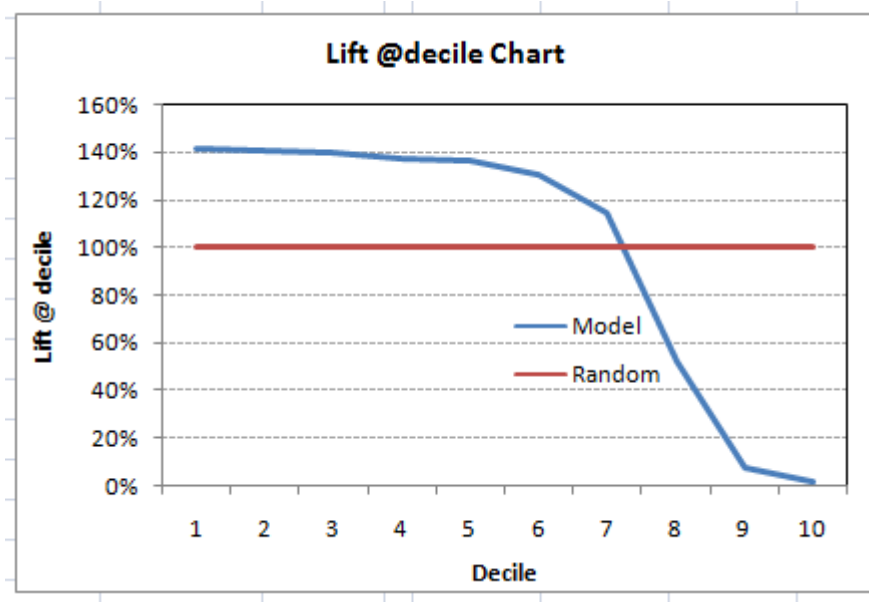
This graph tells you how well is your model segregating responders from non-responders. For example, the first decile, however, has 10% of the population, has 14% of the responders. This means we have a 140% lift at the first decile.

What is the maximum lift we could have reached in the first decile? From the first table of this article, we know that the total number of responders is 3850. Also, the first decile will contain 543 observations. Hence, the maximum lift at the first decile could have been $543/3850 \sim 14.1\%$. Hence, we are quite close to perfection with this model.

Let's now plot the lift curve. The lift curve is the plot between total lift and %population. Note that for a random model, this always stays flat at 100%. Here is the plot for the case in hand:



You can also plot decile-wise lift with decile number:



What does this graph tell you? It tells you that our model does well till the 7th decile. Post which every decile will be skewed towards non-responders. Any model with lift @ decile above 100% till minimum 3rd decile and maximum 7th decile is a good model. Else you might consider oversampling first.

Lift / Gain charts are widely used in campaign targeting problems. This tells us to which decile we can target customers for a specific campaign. Also, it tells you how much response you expect from the new target base.

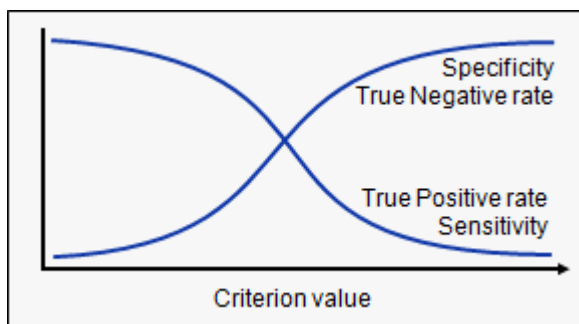
Area Under the ROC Curve (AUC – ROC)

This is again one of the popular evaluation metrics used in the industry. The biggest advantage of using the ROC curve is that it is independent of the change in the proportion of responders. This statement will get clearer in the following sections.

Let's first try to understand what the ROC (Receiver operating characteristic) curve is. If we look at the confusion matrix below, we observe that for a probabilistic model, we get different values for each metric.

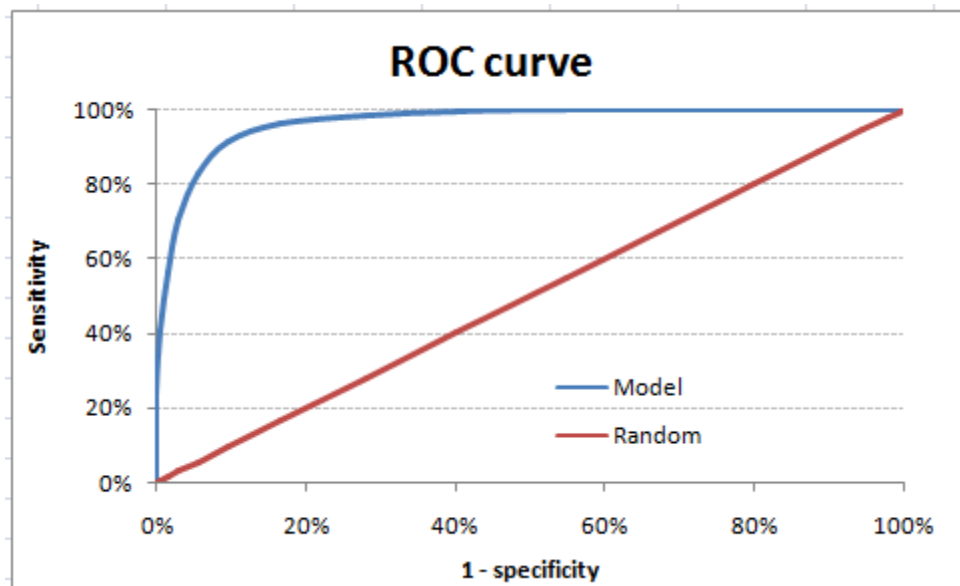
Confusion Matrix		Target			
		Positive	Negative		
Model	Positive	a	b	Positive Predictive Value	$a/(a+b)$
	Negative	c	d	Negative Predictive Value	$d/(c+d)$
		Sensitivity	Specificity	Accuracy = $(a+d)/(a+b+c+d)$	
		$a/(a+c)$	$d/(b+d)$		

Hence, for each sensitivity, we get a different specificity. The two vary as follows:



The ROC curve is the plot between sensitivity and (1- specificity). (1- specificity) is also known as the false positive rate, and sensitivity is also known as the True Positive rate.

Following is the ROC curve for the case in hand.



Let's take an example of threshold = 0.5 (refer to confusion matrix). Here is the confusion matrix:

Target v		
1	0	Grand
3,834	639	
16	951	
3,850	1,590	
99.6%	40.19%	

As you can see, the sensitivity at this threshold is 99.6%, and the (1-specificity) is ~60%. This coordinate becomes on point in our ROC curve. To bring this curve down to a single number, we find the area under this curve (AUC).

Note that the area of the entire square is $1 \times 1 = 1$. Hence AUC itself is the ratio under the curve and the total area. For the case in hand, we get AUC ROC as 96.4%. Following are a few thumb rules:

- $.90-1$ = excellent (A)
- $.80-.90$ = good (B)
- $.70-.80$ = fair (C)
- $.60-.70$ = poor (D)
- $.50-.60$ = fail (F)

We see that we fall under the excellent band for the current model. But this might simply be over-fitting. In such cases, it becomes very important to do in-time and out-of-time validations.

Points to Remember

1. For a model which gives class as output will be represented as a single point in the ROC plot.
2. Such models cannot be compared with each other as the judgment needs to be taken on a single metric and not using multiple metrics. For instance, a model with parameters (0.2,0.8) and a model with parameters (0.8,0.2) can be coming out of the same model; hence these metrics should not be directly compared.
3. In the case of the probabilistic model, we were fortunate enough to get a single number which was AUC-ROC. But still, we need to look at the entire curve to make conclusive decisions. It is also possible that one model performs better in some regions and other performs better in others.

Advantages of Using ROC

Why should you use ROC and not metrics like the lift curve?

Lift is dependent on the total response rate of the population. Hence, if the response rate of the population changes, the same model will give a different lift chart. A solution to this concern can be a true lift chart (finding the ratio of lift and perfect model lift at each decile). But such a ratio rarely makes sense for the business.

The ROC curve, on the other hand, is almost independent of the response rate. This is because it has the two axes coming out from columnar calculations of the confusion matrix. The numerator and denominator of both the x and y axis will change on a similar scale in case of a response rate shift.

Log Loss

AUC ROC considers the predicted probabilities for determining our model's performance. However, there is an issue with AUC ROC, it only takes into account the order of probabilities, and hence it does not take into account the model's capability to predict a higher probability for samples more likely to be positive. In that case, we could use the log loss, which is nothing but a negative average of the log of corrected predicted probabilities for each instance.

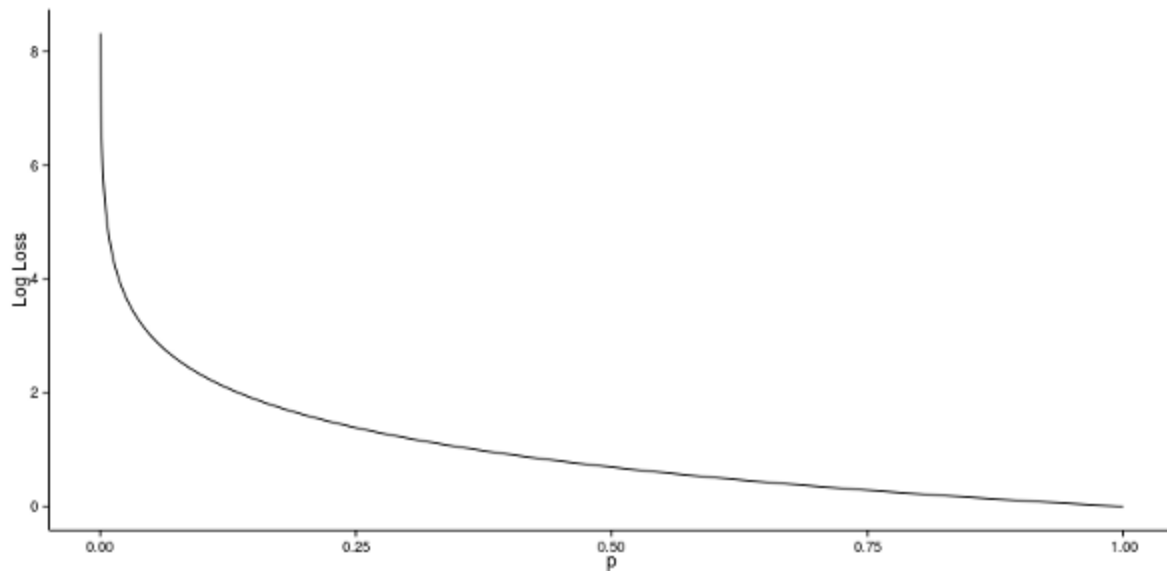
$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

- $p(y_i)$ is the predicted probability of a positive class
- $1-p(y_i)$ is the predicted probability of a negative class
- $y_i = 1$ for the positive class and 0 for the negative class (actual values)

Let us calculate log loss for a few random values to get the gist of the above mathematical function:

- $\text{Log loss}(1, 0.1) = 2.303$
- $\text{Log loss}(1, 0.5) = 0.693$
- $\text{Log loss}(1, 0.9) = 0.105$

If we plot this relationship, we will get a curve as follows:



It's apparent from the gentle downward slope towards the right that the Log Loss gradually declines as the predicted probability improves. Moving in the opposite direction, though, the Log Loss ramps up very rapidly as the predicted probability approaches 0.

So, the lower the log loss, the better the model. However, there is no absolute measure of a good log loss, and it is use-case/application dependent.

Whereas the AUC is computed with regards to binary classification with a varying decision threshold, log loss actually takes the “certainty” of classification into account.

Gini Coefficient

The Gini coefficient is sometimes used in classification problems. The Gini coefficient can be derived straight away from the AUC ROC number. Gini is nothing but the ratio between the area between the ROC curve and the diagonal line & the area of the above triangle.

Following are the formulae used:

$$\text{Gini} = 2 * \text{AUC} - 1$$

Gini above 60% is a good model. For the case in hand, we get Gini as 92.7%.

Concordant – Discordant Ratio

This is, again, one of the most important evaluation metrics for any classification prediction problem. To understand this, let's assume we have 3 students who have some likelihood of passing this year. Following are our predictions:

A – 0.9

B – 0.5

C – 0.3

Now picture this. if we were to fetch pairs of two from these three students, how many pairs would we have? We will have 3 pairs: AB, BC, and CA. Now, after the year ends, we see that A and C passed this year while B failed. No, we choose all the pairs where we will find one responder and another non-responder. How many such pairs do we have?

We have two pairs AB and BC. Now for each of the 2 pairs, the concordant pair is where the probability of the responder was higher than the non-responder. Whereas discordant pair is where the vice-versa holds true. In case both the probabilities were equal, we say it's a tie. Let's see what happens in our case :

AB – Concordant

BC – Discordant

Hence, we have 50% of concordant cases in this example. A concordant ratio of more than 60% is considered to be a good model. This metric generally is not used when deciding how many customers to target etc. It is primarily used to access the model's predictive power.

Decisions like how many to target are again taken by KS / Lift charts.

Root Mean Squared Error (RMSE)

RMSE is the most popular evaluation metric used in regression problems. It follows an assumption that errors are unbiased and follow a normal distribution. Here are the key points to consider on RMSE:

1. The power of 'square root' empowers this metric to show large number deviations.
2. The 'squared' nature of this metric helps to deliver more robust results, which prevent canceling the positive and negative error values. In other words, this metric aptly displays the plausible magnitude of the error term.
3. It avoids the use of absolute error values, which is highly undesirable in mathematical calculations.
4. When we have more samples, reconstructing the error distribution using RMSE is considered to be more reliable.
5. RMSE is highly affected by outlier values. Hence, make sure you've removed outliers from your data set prior to using this metric.
6. As compared to mean absolute error, RMSE gives higher weightage and punishes large errors.

RMSE metric is given by:

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (\text{Predicted}_i - \text{Actual}_i)^2}{N}}$$

where N is the Total Number of Observations.

Root Mean Squared Logarithmic Error

In the case of Root mean squared logarithmic error, we take the log of the predictions and actual values. So basically, what changes are the variance that we are measuring? RMSLE is

usually used when we don't want to penalize huge differences in the predicted and the actual values when both predicted, and true values are huge numbers.

Root Mean Squared Error (RMSE)	Root Mean Squared Log Error (RMSLE)
$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	$\sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2}$
predictionactual	

1. If both predicted and actual values are small: RMSE and RMSLE are the same.
2. If either predicted or the actual value is big: RMSE > RMSLE
3. If both predicted and actual values are big: RMSE > RMSLE (RMSLE becomes almost negligible)

R-Squared/Adjusted R-Squared

We learned that when the RMSE decreases, the model's performance will improve. But these values alone are not intuitive.

In the case of a classification problem, if the model has an accuracy of 0.8, we could gauge how good our model is against a random model, which has an accuracy of 0.5. So the random model can be treated as a benchmark. But when we talk about the RMSE metrics, we do not *have* a benchmark to compare.

This is where we can use the R-Squared metric. The formula for R-Squared is as follows:

$$R^2 = 1 - \frac{\text{MSE}(\text{model})}{\text{MSE}(\text{baseline})}$$

$$\frac{\text{MSE}(\text{model})}{\text{MSE}(\text{baseline})} = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (\bar{y} - \hat{y}_i)^2}$$

MSE(model): Mean Squared Error of the predictions against the actual values

MSE(baseline): Mean Squared Error of mean prediction against the actual values

In other words, how good is our regression model as compared to a very simple model that just predicts the mean value of the target from the train set as predictions?

Adjusted R-Squared

A model performing equal to the baseline would give R-Squared as 0. Better the model, the higher the r^2 value. The best model with all correct predictions would give R-Squared of 1.

However, on adding new features to the model, the R-Squared value either increases or

remains the same. R-Squared does not penalize for adding features that add no value to the model. So an improved version of the R-Squared is the adjusted R-Squared. The formula for adjusted R-Squared is given by:

$$\bar{R}^2 = 1 - \left(1 - R^2\right) \left[\frac{n-1}{n-(k+1)} \right]$$

k: number of features

n: number of samples

As you can see, this metric takes the number of features into account. When we add more features, the term in the denominator $n-(k+1)$ decreases, so the whole expression increases.

If R-Squared does not increase, that means the feature added isn't valuable for our model.

So overall, we subtract a greater value from 1 and adjusted r^2 , in turn, would decrease.

Beyond these 12 evaluation metrics, there is another method to check the model performance. These 7 methods are statistically prominent in data science. But, with the arrival of machine learning, we are now blessed with more robust methods of model selection. **Yes! I'm talking about Cross Validation.**

Though cross-validation isn't really an evaluation metric that is used openly to communicate model accuracy, the result of cross-validation provides a good enough intuitive result to generalize the performance of a model.

Let's now understand cross-validation in detail.

Cross Validation

Let's first understand the importance of cross-validation. Due to my busy schedule these days, I don't get much time to participate in data science competitions. A long time back, I participated in TFI Competition on Kaggle. Without delving into my competition performance, I would like to show you the dissimilarity between my public and private leaderboard scores.

Here Is an Example of Scoring on Kaggle!

For the TFI competition, the following were three of my solution and scores (the lesser, the better):

Submission	Files	Public Score	Private Score	Selected?
Mon, 04 May 2015 12:59:31 Edit description	submission_all_with_sai3.csv	1649776.86428	1809956.02878	☒
Mon, 04 May 2015 11:48:54 Edit description	submission_all_wit	1651071.47287	1802503.24607	☐
Mon, 13 Apr 2015 13:28:08 Edit description	submission_all.csv	1677138.71291	1795007.23155	☒

You will notice that the third entry which has the worst Public score turned out to be the best model on Private ranking. There were more than 20 models above the “submission_all.csv”, but I still chose “submission_all.csv” as my final entry (which really worked out well). What caused this phenomenon? The dissimilarity in my public and private leaderboards is caused by over-fitting.

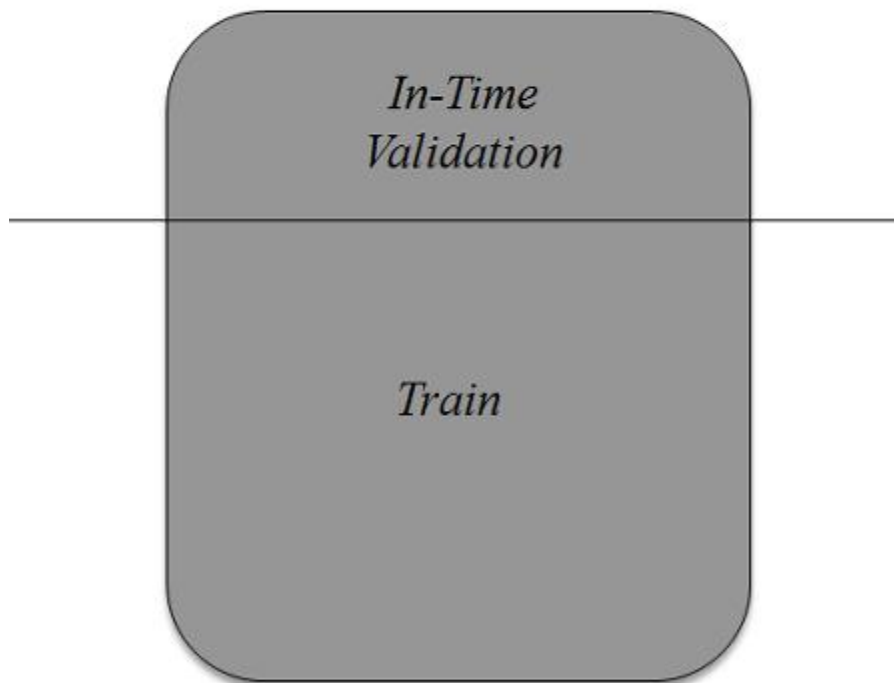
Over-fitting is nothing, but when your model becomes highly complex that it starts capturing noise, also. This ‘noise’ adds no value to the model but only inaccuracy.

In the following section, I will discuss how you can know if a solution is an over-fit or not before we actually know the test set results.

The Concept of Cross-Validation

Cross Validation is one of the most important concepts in any type of data modeling. It simply says, try to leave a sample on which you do not train the model and test the model on this sample before finalizing the model.

Training Population



The above diagram shows how to validate the model with the in-time sample. We simply divide the population into 2 samples and build a model on one sample. The rest of the population is used for in-time validation.

Could there be a negative side to the above approach?

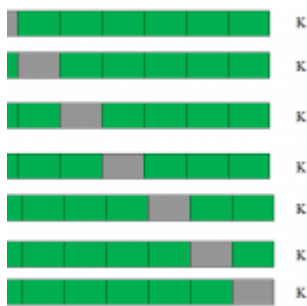
I believe a negative side of this approach is that we lose a good amount of data from training the model. Hence, the model is very high bias. And this won't give the best estimate for the coefficients. So what's the next best option?

What if we make a 50:50 split of the training population and the train on the first 50 and validate on the rest 50? Then, we train on the other 50 and test on the first 50. This way, we

train the model on the entire population, however, on 50% in one go. This reduces bias because of sample selection to some extent but gives a smaller sample to train the model on. This approach is known as 2-fold cross-validation.

K-Fold Cross-Validation

Let's extrapolate the last example to k-fold from 2-fold cross-validation. Now, we will try to visualize how a k-fold validation work.



This is a 7-fold cross-validation.

Here's what goes on behind the scene: we divide the entire population into 7 equal samples. Now we train models on 6 samples (Green boxes) and validate on 1 sample (grey box). Then, at the second iteration, we train the model with a different sample held as validation. In 7 iterations, we have basically built a model on each sample and held each of them as validation. This is a way to reduce the selection bias and reduce the variance in prediction power. Once we have all 7 models, we take an average of the error terms to find which of the models is best.

How does this help to find the best (non-over-fit) model?

k-fold cross-validation is widely used to check whether a model is an overfit or not. If the performance metrics at each of the k times modeling are close to each other and the mean of

the metric is highest. In a Kaggle competition, you might rely more on the cross-validation score than the Kaggle public score. This way, you will be sure that the Public score is not just by chance.

Classification Error

Minimum classification error (MCE) is a measure of the quality of a classification model in artificial intelligence and machine learning. This measure refers to the lowest error rate that can be achieved when classifying data from a test set.

The MCE is used to assess the ability of a classification model to generalize to new and unseen data, known as generalizability. A model with a low MCE is able to correctly classify most test data and has better generalization ability than a model with a high MCE.

The MCE is determined by comparing the model predictions with the actual labels of the test data. Classification error is defined as the proportion of examples that are misclassified. The MCE is achieved when the minimum possible classification error value is found for the model, implying that the model is as accurate as possible in the classification task.

The MCE is an important measure in the development and evaluation of classification models in artificial intelligence and machine learning, as it allows comparing the quality of different models and selecting the best one for a specific task. In addition, the MCE can help identify areas where the model needs improvement to improve its generalization capability.

It can also be considered as a variant of the LVQ (Learning Vector Quantization) method. In this sense, MCE is a training technique that uses the minimum classification error criterion to adjust the weights of the coding vectors in the LVQ network. The goal of MCE is to minimize the classification error rate, i.e., the proportion of incorrectly classified samples. MCE uses a cost function that measures the discrepancy between the network output and the expected value of the output for each training sample.

MCE is used in binary and multiclass classification problems, and is useful when the number of training samples is limited or when the classes are unbalanced.